

Proving the Program You Run

Mutable borrows inside a dependent type theory, by observation and closing off

Abstract. Verified systems code today is written twice: once as the efficient imperative program that runs, and once as a pure model that the proofs are about, joined by a refinement proof or a translation. We present Ochr, a core calculus in which the two are the same text. Ochr combines Rust-style mutable borrows with a Lean-style dependent type theory, and its definitional equality unfolds in-place code as Lean’s unfolds pure code. Three ideas make this work. First, an *observational* equality **Id A t u** compares two computations by what they return and what they leave behind in the places they may write; it is not a new primitive but computes to ordinary equality between observations. Second, a call whose body is stuck on an unknown value is *closed off*: its result, and the final content of every place it borrowed, become *sealed programs*, closed source programs that own their state. Sealed programs are the backward functions of Aeneas, but written in the source language, so the programmer never sees a second language. Third, induction hypotheses are typed in the caller’s environment, where the borrow structure around the recursive argument performs the congruence step that a pure proof would write by hand. Proofs about in-place functions become plain structural recursion. We give the calculus with a complete formal definition; worked examples on numbers, lists and binary search trees, including a function that returns a borrow into its argument; a model in which Aeneas’s backward functions reappear, reducing consistency to a set-theoretic interpretation in the style of Lean’s; an executable Lean implementation of the checker whose test suite records, for every side condition, the false proof it rules out; and a mechanisation of the central lemmas of the effectful layer for a first-order fragment.

1 Introduction

Here is an addition on Peano numerals that allocates nothing. It walks down the successors of `*x` until it reaches the final `Z` and moves `y` into that spot:

```
AddM(x : &Nat, y : Nat) : Unit by x :=  
  match *x { Z => *x := y | S p => AddM(&p, y) }
```

`&Nat` is a mutable borrow in the sense of Rust: a unique, temporary right to read and write a `Nat` owned by someone else. In the successor case the pattern variable `p` names the predecessor field in place, and `&p` reborrows it for the recursive call. Here is a theorem about `AddM`, stated and proved in the same language:

```
AddMZero(x : &Nat) : Id Unit (AddM(x, 0)) () by x :=  
  match *x { Z => refl | S p => AddMZero(&p) }
```

The statement says that running `AddM(x, 0)` is indistinguishable from doing nothing: both return `()`, and both leave the number behind `x` as it was. The proof is structural recursion and nothing else. There is no pure model of `AddM`, no refinement relation, no loop invariant, no separation-logic assertion and no translation into another language. The checker accepts the recursive call as a proof of the successor case by running both sides of the equation on a symbolic input and comparing what they leave behind.

This paper is about the type theory that makes that possible.

1.1 The two-language problem

Verified software is usually built in one of two ways. In the first, everything is written in a pure proof assistant such as Lean or Coq, and performance comes from the compiler’s ability to turn functional updates into in-place ones where it can prove the old value is dead. In the second, the efficient program is written in an imperative language, and a separate pure specification is related to it by a refinement proof: Verus [15] and Creusot [9]

check Rust against specifications discharged by SMT, VeriFast checks C against separation-logic contracts, and Aeneas [12] translates safe Rust into a pure Lean program about which the user then proves theorems.

Both routes keep two artifacts. In the second route this is visible: there is a program and there is a specification, and the proof is about their relationship. In the first route it is hidden but present: the program the proof is about is not the program that runs, and the gap is bridged by an optimisation the programmer cannot state or rely on.

Aeneas comes closest to closing the gap. Its insight is that Rust’s ownership discipline makes mutation equivalent to state passing: a function taking $x : \&\text{mut } T$ is a pure function returning, besides its result, a *backward function* that computes the final value of $*x$. Aeneas performs this translation once, eagerly, for a whole program, and the user proves theorems about the translated Lean code, where the backward functions appear as ordinary definitions. The user therefore works in two languages at once, and must understand how one maps onto the other. We show that this translation can be performed **inside the type checker, lazily, as definitional equality, and in the source language itself.**

1.2 Three ideas

Observational equality. The type $\text{Id } A \ t \ u$ compares two computations t and u of result type A , run from the current state. It holds when they return equal values and leave equal contents in every place they may write. Id is not a new primitive with its own introduction and elimination rules. At a given environment it computes to an ordinary propositional equality between two *observations*, the tuples of result and final contents, and that equality computes further by the structure of the data in the manner of observational type theory [5]. The places a computation may write are read off its syntax, so they are an output of the computation like its result. This is the sense in which Ochr has no pure/impure divide: any program may appear in a type, and what it does to its environment is part of what the type talks about.

Closing off. Definitional equality in Ochr is normalisation by a deterministic symbolic machine: two terms are equal when the machine sends them to the same normal form. On symbolic inputs the machine gets stuck, and a dependent type theory needs a name, a *neutral term*, for every stuck computation. A stuck pure computation names itself: Lean’s $\text{Nat.add } n \ 0$ is its own normal form. A stuck effectful call does not, because part of what it produces lives in the places it borrowed. We *close off* such a call: its result and the final content of each borrowed place are replaced by *sealed programs*, closed source programs that own their state. For AddM on an unknown σ , the borrowed place receives $[\text{let } c = \sigma; \text{AddM}(\&c, \ 0); \ c]$, which is the pure wrapper a programmer would write anyway. When a later case split refines σ , the sealed program runs again. Sealed programs are exactly Aeneas’s backward functions, but they are written in the source language and generated on demand, so the programmer never writes or reads a second language. A function that returns a borrow is closed off into a sealed program with a hole awaiting the final value of the returned borrow; this is Aeneas’s region abstraction, in the same form.

The environment does the congruence. When the checker types the recursive call $\text{AddMZero}(\&p)$, it evaluates the callee’s statement in the caller’s environment, where $\&p$ points into the predecessor field of the caller’s number. The observation therefore sees the successor that surrounds the recursive call, and the induction hypothesis arrives already wrapped in S : it is literally the goal. The reasoning that a proof about a state-passing translation must do by hand, relating the callee’s statement about *its* borrowed place to the caller’s statement about the caller’s own data, has been performed by evaluation; for natural numbers it amounts to the step $\text{cong } S \text{ ih}$, and in general it is a frame argument. Soundness rests on the frame property that Rust’s ownership discipline provides: a call can affect only what it is passed.

1.3 Why this is not impossible

Pédrot and Tabareau’s fire triangle [24] shows that a type theory cannot combine substitution of arbitrary terms, dependent elimination and observable effects without becoming inconsistent. Their notion of an observable effect concerns closed terms, and ownership removes it: a closed Ochr program can mutate only places it created itself, so it always behaves like a value. The danger reappears for *open* terms, whose effects on the environment are observable, and there it takes exactly the form they describe, a desynchronisation between the effects performed in

a term and in its type. A statement is evaluated once through the stuck-computation machinery, at a definition’s generic call, and again directly, at every instance, and any step on which the two paths disagree is a proof of false; we found several such steps while designing the calculus (Section 6). Ochr keeps the two paths synchronised by one discipline: a type contains only values, a computation mentioned in a type contributes only its observation, run on a private copy of the environment, and every decision the two paths must agree on (which terms are erased, what a stuck block captures, which loans a match ends) is made from syntax rather than from normal forms. Unlike the call-by-value corner of their analysis, this keeps dependent `let` and large elimination, because typing threads the values computed so far.

1.4 Contributions

- A core calculus, Ochr, combining mutable borrows with a dependent type theory with a universe of proof-irrelevant propositions, whose definitional equality unfolds imperative code (Section 3, Section 4).
- *Closing off*, which gives every stuck effectful call a neutral form made of sealed source programs, including calls that return borrows (Section 4).
- An observational equality between computations that is derived rather than primitive (Section 5).
- Worked examples, among them the equivalence of `AddM` with a version that first obtains a borrow of the final node and then writes through it, each proved by bare structural recursion (Section 2).
- A model in which the backward functions reappear: computations translate into Lean’s type theory, and types into a set-theoretic interpretation in the style of Carneiro’s model of Lean, giving consistency relative to ZFC with inaccessibles (on paper); an adequacy theorem relating the symbolic machine to concrete evaluation; and a stability lemma saying why the checker’s two evaluation paths agree (Section 7). Parts of the effectful layer are mechanised for a first-order fragment.
- An executable implementation of the checker in Lean 4 that checks every example in the paper (Section 8).

2 Ochr by example

This section introduces Ochr through its running examples. Everything is informal here; Section 3 onwards makes it precise.

2.1 Imperative code is evaluated inside types

Ochr programs are made of definitions with parameters and a body. Borrow types `&T` are Rust’s mutable references; places are variables `x`, dereferences `*p` and the predecessor field `p.1` of a number. A `match` on a place binds its pattern variable to a *sub-place*, not a copy, so in `match *x { Z => ... | S p => ... }` the name `p` stands for the field `(*x).1` and `&p` reborrows it.

`Add` wraps `AddM` behind a pure interface. It owns its first argument, lends it to `AddM`, and returns it:

```
Add(x : Nat, y : Nat) : Nat := AddM(&x, y); x
```

Because definitional equality is evaluation, `Id Nat (Add(2, 3)) 5` holds by `refl`: the checker runs `Add(2, 3)`, which runs `AddM` in place on a local copy, and compares the result with 5. There is nothing special about imperative code here, and nothing special about `Add` being “pure”: it is simply a program whose observable output is its result.

The machine that does this is the symbolic semantics of Aeneas’s low-level borrow calculus [12]. An environment Ω maps variables to values; borrowing `x` moves its content into the borrow and leaves a *loan* behind; ending the borrow moves the content back. Running `Add(2, 3)` passes through these states:

```
{ x ↦ 2 }                               // enter Add
{ x ↦ loan₀ } ⊢ AddM(borrow₀ 2, 3)        // &x: the content moves into the borrow
{ x ↦ loan₀ | x' ↦ borrow₀ (S loan₁), ... } // AddM matched S and reborrowed the field
...
{ x ↦ 5 }                               // borrows ended, contents returned
```

2.2 Symbolic inputs, and what a stuck call leaves behind

To reason about all numbers at once, the checker runs programs on *abstract values* σ , which play the role of Lean’s free variables. Running $\text{Add}(\sigma, \theta)$ borrows $x \mapsto \sigma$ and calls $\text{AddM}(\text{borrow}_\theta \sigma, \theta)$, whose body immediately gets stuck: it must match on σ .

In a pure type theory a stuck term is its own normal form. Here the call has also produced something that lives in the environment, namely the final content of the place it borrowed, and that content needs a name. We *close off* the call: the call returns, and the loan it held is filled with a *sealed program*, a closed source program that owns its state and computes that content:

```
{ x ↦ loanθ } ⊢ AddM(borrowθ σ, θ)
→ { x ↦ [let c = σ; AddM(&c, θ); c] } ⊢ ()
```

Write $N(\sigma)$ for $[let\ c = \sigma; \text{AddM}(\&c, \theta); c]$. It is a neutral value, like a stuck `Nat.rec` in Lean, and it is literally the body of $\text{Add}(\sigma, \theta)$: the sealed program *is* the pure wrapper, generated on demand. It is also exactly the value that Aeneas’s backward function for AddM would compute [12], written without leaving the source language. When a later case split refines σ to $S\ \sigma'$, $N(S\ \sigma')$ runs again, now makes progress through one successor, closes off the inner recursive call, and normalises to $S\ N(\sigma')$.

2.3 Equality between computations

The type `Id Unit (AddM(x, θ)) ()` compares two computations. It is evaluated at the environment where it is formed. Each side runs on its own copy of that environment; the result and the final contents of the places the side may write form its *observation*; and `Id` computes to ordinary equality between the two observations. For a definition taking $x : \&\text{Nat}$, the checker works at the definition’s *generic call* $\text{AddMZero}(\&c)$ from $\{ c \mapsto \sigma \}$, so the place that may be written is `c`, the owner of x ’s loan:

```
[[AddM(x, θ)]] = ((), N(σ))      // the call is stuck, so c receives the sealed program
[()]          = ((), σ)         // nothing happens; ending x's borrow returns σ to c

Id Unit (AddM(x, θ)) () ≡ Eq (Unit × Nat) ((), N(σ)) ((), σ) ≡ Eq Nat N(σ) σ
```

The last step uses the observational computation rules for equality [5]: equality of pairs is a conjunction, reflexive equations are τ , and τ is a unit for $\wedge$. The statement “running $\text{AddM}(x, \theta)$ has no effect” has become the ordinary proposition that $N(\sigma)$ equals σ .

2.4 The environment performs the congruence

Now the proof:

```
AddMZero(x : &Nat) : Id Unit (AddM(x, θ)) () by x :=
  match *x { Z => refl | S p => AddMZero(&p) }
```

The match on $*x$ splits on σ . In the `Z` branch the goal is $\text{Eq Nat } N(Z)\ Z$; $N(Z)$ runs to `Z`, the equation is reflexive and so τ , and `refl` proves it. In the `S` branch the goal is $\text{Eq Nat } N(S\ \sigma')\ (S\ \sigma')$, which normalises to $\text{Eq Nat } (S\ N(\sigma'))\ (S\ \sigma')$.

The recursive call is checked by the same rule that produced the goal: the callee’s statement is evaluated at the call site, with its parameter bound to the argument. At the call site the environment is

```
{ c ↦ loanθ | x ↦ borrowθ (S loan1) } and the argument is borrow1 σ'
```

because matching `S p` through the borrow and taking `&p` leaves the successor in place, around a loan for the predecessor. The owner of the argument’s loan is found by following it outwards: `loan1` sits inside x ’s borrow, whose loan sits in `c`. So the recursive call’s statement observes `c`, and running AddM on the argument fills `loan1` with $N(\sigma')$, after which `c` holds $S\ N(\sigma')$:

```

[[AddM(x', 0)]] at the call site = ((), S N(σ'))
[[()]] at the call site = ((), S σ')
AddMZero(&p) : Eq Nat (S N(σ')) (S σ')

```

This is the goal, symbol for symbol. The successor that a pure proof would add with `cong S` was supplied by the environment. The recursive call is a proof, and proofs are erased at runtime, so the checker does not run it: its borrow argument is returned unchanged.

For comparison, here is the same theorem about the pure wrapper:

```

AddZero(x : Nat) : Id Nat (Add(x, 0)) x :=
  match x { Z => refl | S p => AddMZero(&p) }

```

`Add(x, 0)` writes nothing outside itself, so its observation is just its result, and the goal is again `Eq Nat N(σ)` σ , the very proposition `AddMZero`'s statement computed to; in the successor case it is `Eq Nat (S N(σ')) (S σ')`. The proof borrows the predecessor field of the *owned* number x and appeals to the in-place lemma. The environment again supplies the successor, so even the theorem about the pure function needs no congruence step. An induction hypothesis about a *copy* of the predecessor, `AddZero(p)`, would have type `Eq Nat N(σ') σ'`, and the successor would have to be added by hand with a congruence lemma derived from `J`. The two theorems are interchangeable: `AddZero(x) := AddMZero(&x)` type-checks, because both statements normalise to the same proposition.

2.5 Returning a borrow

A more idiomatic in-place addition first finds the final node and then writes through it:

```

TailM(x : &Nat) : &Nat by x := match *x { Z => x | S p => TailM(&p) }
AddM'(x : &Nat, y : Nat) : Unit := let t = TailM(x); *t := y

```

`TailM` returns a borrow into its argument. When it is stuck on σ , the place it borrowed cannot be given its final content yet, because that depends on what the caller will later write through the returned borrow. Closing off such a call returns a borrow of a fresh k and fills the argument's loan with a sealed program that has a *hole* `loan_k` for the returned borrow's final content:

```

{ c ↦ loan0 } ⊢ TailM(borrow0 σ)
  → { c ↦ [let c = σ; let r = TailM(&c); *r := loan_k; c] }
    ⊢ borrow_k [let c = σ; let r = TailM(&c); *r]

```

In `AddM'`, the caller writes y through t and drops it; ending `borrow_k` substitutes the final content y for the hole. What remains in c is `[let c = σ; let r = TailM(&c); *r := y; c]`: the effect of `AddM'`, as a program. The hole is the neutral form of an Aeneas region abstraction, and of a RustHorn prophecy [19]: it records only the *final* value written through the returned borrow.

The equivalence of the two additions is proved by the same bare recursion:

```

AddMEq(x : &Nat, y : Nat) : Id Unit (AddM(x, y)) (AddM'(x, y)) by x :=
  match *x { Z => refl | S p => AddMEq(&p, y) }

```

In the successor branch, re-running `AddM'`'s sealed program on `S σ'` unfolds `TailM` once, closes off its inner call with a fresh hole, and the pending write `*r := y` then fills that hole; on the other side the recursive call's statement, evaluated at the call site, performs the same steps through the caller's borrow. Both sides arrive at `Eq Nat (S A(σ', y)) (S B(σ', y))`, where A and B are the sealed programs for the two additions on the predecessor. The theorem for an owned number follows by lending it: `AddMEqOwned(x : Nat) : Id Unit (AddM(&x, 0)) (AddM'(&x, 0)) := AddMEq(&x, 0)`.

2.6 Proofs about the current state

Dependent types are not only for stating theorems after the fact. A function can demand, as an argument, a proof about the *current* contents of a borrow. Here is in-place subtraction, which is total only when the subtrahend is no larger than the number:

```
Le(a : Nat, b : Nat) : Prop by a :=
  match a { Z =>  $\top$  | S a' => match b { Z => Eq Nat Z (S Z) | S b' => Le(a', b') } }

SubM(x : &Nat, y : Nat, h : Le(y, *x)) : Unit by y :=
  match y { Z => () | S q => match *x { Z => () | S p => *x := p; SubM(x, q, h) } }
```

The type of `h` mentions `*x`, the content of the borrow at the moment of the call. Inside `SubM`, after `*x := p` has overwritten that content, the recursive call needs a proof of `Le(q, *x)` about the *new* content; the old hypothesis `h`, of type `Le(S q, S p)`, provides it, because that type was formed when `h` was bound and normalises to `Le(q, p)`.

Now a caller that first adds and then subtracts:

```
LeAdd(n : Nat, m : Nat) : Le(n, Add(n, m)) by n := match n { Z => refl | S n' => LeAdd(n', m) }

AddSub(x : &Nat, y : Nat) : Unit := let old = *x; AddM(&*x, y); SubM(x, old, LeAdd(old, y))
```

At the call to `SubM`, `*x` has just been mutated in place by `AddM`, so on symbolic input it holds the sealed program $N(\sigma, y) = [\text{let } c = \sigma; \text{AddM}(\&c, y); c]$, and `SubM` demands a proof of `Le( $\sigma$ ,  $N(\sigma, y)$ )`. The lemma `LeAdd` is about the *pure* `Add` of the snapshot `old`, and its type is `Le( $\sigma$ , Add( $\sigma$ , y))`. The two meet because `Add( $\sigma$ , y)` normalises to the very same sealed program: the in-place computation and the pure one are the same program to the type checker. A proof about the pure function is accepted where a proof about the mutated state is required, with no bridging lemma. Using the snapshot after further mutation, `...; AddM(&*x, y); *x := Z; SubM(x, old, LeAdd(old, y))`, is rejected, since the requirement then mentions `Z`.

Finally, a theorem about the whole: adding `y` and then subtracting the old value leaves exactly `y`.

```
AddSubId(x : &Nat, y : Nat) : Id Unit (AddSub(x, y)) (*x := y) by x :=
  match *x { Z => refl | S p => let c = p; AddSubId(&c, y) }
```

Here the successor case copies the predecessor into a fresh place `c` instead of borrowing it in place. Borrowing would supply the surrounding `S` to the induction hypothesis, as in `AddMZero`, and the goal has no `S` to match: `AddSub` peels the successor off. Borrowing supplies the congruence and copying withholds it, and the programmer chooses.

2.7 Trees

Nothing above is specific to numbers. With several constructors and several fields, a pattern variable names a field place, and the environment keeps every field that a recursive call does not touch. In-place insertion into a binary search tree recurses into the left or the right subtree according to a comparison, so which place is mutated depends on a value:

```
InsertM(t : &Tree, k : Nat) : Unit by t :=
  match *t { Leaf          => *t := Node(Leaf, k, Leaf)
          | Node(l, v, r) => let b = Lt(k, v);
                           match b { True => InsertM(&l, k) | False => InsertM(&r,
k) } }

InsertMEq(t : &Tree, k : Nat) : Id Unit (InsertM(t, k)) (*t := Insert(*t, k)) by t :=
  match *t { Leaf          => refl
```

```

| Node(l, v, r) => let b = Lt(k, v);
                    match b { True => InsertMEq(&l, k) | False => InsertMEq(&r,
k) } }

```

Insert is the pure insertion that rebuilds the path. In the **False** case, both the goal and the induction hypothesis, evaluated at the call site, observe the whole tree as **Node**(σ_l , σ_v , [...]): the fields l and v are carried by the environment, and only the right subtree differs, as in-place insertion on one side and pure insertion on the other. The proof is bare recursion again. The comparison **Lt**(σ_k , σ_v) is itself stuck, so the split on b is a split on a sealed program: the checker names its value by a fresh abstract value and replaces every derivation of the same closed program by it, including those produced later when the goal's sealed programs run again. A theorem about a measure mixes the two styles: **Size**(**Insert**(t , k)) is **S**(**Size**(t)) given one arithmetic lemma, $x + S\ y = S\ (x + y)$, which is proved in place by bare recursion and transferred to the pure **Add** by lending, as **AddZero** was.

2.8 Branching

A **match** on an abstract value in the middle of a function cannot pick an arm. Each arm is checked separately, and the rest of the function is then checked once, from the state in which the match itself has been closed off like a call. This covers borrows whose origin depends on the branch:

```

AddToOne(b : Nat, x1 : &Nat, x2 : &Nat, y : Nat) : Unit :=
  let r = match b { Z => x1 | S _ => x2 }; AddM(r, y)

```

The closed-off match returns a borrow with a hole that appears in the sealed programs for both x_1 's and x_2 's places; whichever the match would have chosen receives the final content. A proof about **AddToOne** splits on b , after which the sealed programs run and the goal becomes a statement about **AddM** alone:

```

AddToOneZero(b : Nat, x1 : &Nat, x2 : &Nat) : Id Unit (AddToOne(b, x1, x2, 0)) () :=
  match b { Z => AddMZero(x1) | S _ => AddMZero(x2) }

```

The footprint contains the owners of both borrows. In the **Z** arm the observation of x_2 's owner is unchanged on both sides, so its equation is reflexive and computes to $\top$, and what remains is exactly **AddMZero**(x_1)'s statement.

3 The calculus

Ochr is a dependent type theory in the style of Lean's kernel [8], extended with places, mutable borrows and sequencing. This section gives its syntax and the runtime structures of its evaluator. We present the rules for natural numbers and the unit type, plus pairs and the propositional connectives needed by equality; user-declared inductive types with several constructors and fields change only the grammar of patterns and places, and Section A gives the general definition.

3.1 Syntax

$ \begin{aligned} t, u, A, B &:: x \mid \text{Prop} \mid \text{Type}_i \\ & \mid \Pi(x_1 : A_1 \dots x_n : A_n). B \mid \text{fix } f(x_1 : A_1 \dots x_n : A_n) : B \text{ by } x_j := t \mid t(u_1, \dots, u_n) \\ & \mid \text{Nat} \mid \text{Z} \mid \text{S } t \mid \text{Unit} \mid () \mid A \times B \mid (t, u) \mid t.1 \mid t.2 \\ & \mid \text{Eq } A\ t\ u \mid \text{refl} \mid J(A, a, b, P, h, t) \mid \top \mid P \wedge Q \mid \langle h, k \rangle \\ & \mid \&A \mid p \mid \&p \mid p := t \mid \text{let } x = t; u \mid \text{let } x : A = t; u \mid t; u \\ & \mid \text{match } p \{ Z \Rightarrow t \mid S\ y \Rightarrow u \} \\ & \mid \text{ld } A\ t\ u \\ p &:: x \mid *p \mid p.1 \end{aligned} $	variables, universes functions, calls data propositions places and borrows case analysis computation equality places
--	--

Figure 1: Syntax of Ochr. **Eq**, $\top$ and $\wedge$ live in **Prop**. A **fix** without **by** is an ordinary λ .

Figure 1 gives the syntax. Types and terms share one grammar, as in any pure type system. Functions are n-ary and calls are saturated: a partial application would be a closure capturing its arguments, and a closure capturing a borrow is outside the core. A recursive function names the parameter it recurses on.

The imperative fragment is small. A *place* p is a variable, a dereference $*p$, or the predecessor field $p.1$ of a number. A place used as a term reads it; $\&p$ borrows it; $p := t$ assigns it; $\text{let } x = t; u$ introduces a new place x . The pattern variable of a *match* is a *sub-place*: in $\text{match } p \{ S \ y \Rightarrow u \}$, y stands for $p.1$, and nothing is copied.

$\&A$ is the type of a mutable borrow of an A . In the core, borrow types occur only as the types of variables, parameters and results, never inside another type: there are no borrows stored in data structures and no borrows of borrows. There are no shared borrows and no loops. Recursion is structural.

The propositional fragment is Lean’s: **Prop** is an impredicative universe with definitional proof irrelevance, erased at runtime, and **Eq** is its equality. $\text{Id } A \ t \ u$ compares two *computations* t and u of type A ; Section 5 shows that it is not a new primitive.

3.2 Values and environments

$v, w ::=$	$Z \mid S \ v \mid () \mid (v, w) \mid \star \mid \text{closures} \mid \text{types}$	data, proofs, closures, types
	$\mid \text{borrow}_\ell v \mid \text{loan}_\ell \mid \perp$	borrows, loans, moved-out
	$\mid n$	neutrals
$n ::=$	$\sigma \mid [t]$	abstract values, sealed programs
$\Omega ::=$	$\cdot \mid \Omega, x : A \mapsto v \mid \Omega \mid \Omega'$	bindings, grouped in frames

Figure 2: Runtime structures.

The evaluator manipulates the values of Figure 2. Following the low-level borrow calculus of Aeneas [12], a borrow carries the borrowed content with it: $\text{borrow}_\ell v$ is a borrow, identified by ℓ , whose current content is v , and loan_ℓ is the placeholder left at the place it was taken from. When the borrow ends, its content is put back where the loan is. We depart from Aeneas in one respect: **a loan is a variable bound by its borrow**. It may occur anywhere inside a value, including inside a sealed program, and inside sealed programs it may occur more than once; ending the borrow substitutes its content for every occurrence. $\perp$ marks a place whose content has been moved out, and $\star$ is the value of a proof.

Neutral values are stuck computations. An *abstract value* σ is an unknown value of a known type, playing the role of a free variable in Lean’s kernel: the checker introduces one for every parameter of a definition it checks, and nowhere else. A *sealed program* $[t]$ is a closed program whose run is stuck on an abstract value. Sealed programs are produced by closing off stuck calls (Section 4) and are the only neutral form besides abstract values. A value is *borrow-free* if it contains no borrow and no live loan.

An environment Ω is a stack of frames, one per active call, each binding variables to values. $\text{content}(\Omega, p)$ follows a place through variables, borrows ($*p$ looks inside $\text{borrow}_\ell v$ at v) and successors ($p.1$ looks inside $S \ v$ at v). We write $\Omega[p \mapsto v]$ for the environment with the content of p replaced.

4 Evaluation and closing off

Definitional equality in Ochr is normalisation by one deterministic machine. With effects, rewriting-anywhere is not confluent: evaluating $(\lambda y. (x := 2; y))(x := 1)$ argument-first and substituting-first leave x holding different numbers. A fixed call-by-value order restores uniqueness, and the machine then plays the role that normalisation by evaluation plays for pure type theories [2]: two terms are definitionally equal when the machine sends them to the same normal form. On concrete inputs it is an ordinary interpreter. On abstract inputs it runs until it has to inspect an abstract value, and the question the rest of this section answers is what it returns then.

The judgement is big-step: $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$ runs t from environment Ω to value v and environment Ω' , and a run may instead be *stuck* (it needs to inspect a neutral) or fail with a borrow error.

4.1 Borrows

$$\begin{array}{c}
\frac{\text{borrow}_\ell v \text{ in } \Omega}{\Omega \rightsquigarrow \Omega[\text{borrow}_\ell v := \perp][v/\text{loan}_\ell]} \text{ END} \quad \frac{v = \text{content}(\Omega, p)}{\langle \Omega, \&p \rangle \Downarrow \langle \Omega[p \mapsto \text{loan}_\ell], \text{borrow}_\ell v \rangle} \text{ BORROW} \\
\\
\frac{v = \text{content}(\Omega, p) \quad \text{borrow-free}}{\langle \Omega, p \rangle \Downarrow \langle \Omega, v \rangle} \text{ READ} \\
\\
\frac{\text{content}(\Omega, p) = \text{borrow}_\ell v}{\langle \Omega, p \rangle \Downarrow \langle \Omega[p \mapsto \perp], \text{borrow}_\ell v \rangle} \text{ MOVE} \quad \frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \Omega_2 = \text{drop}(\Omega_1, \text{content}(\Omega_1, p))}{\langle \Omega, p := t \rangle \Downarrow \langle \Omega_2[p \mapsto v], () \rangle} \text{ ASSIGN}
\end{array}$$

Figure 3: Borrows. Every rule that accesses a place p is preceded by $[\text{Access}]$: end every borrow whose loan lies on the path to p or inside its content (for a match, which inspects only the head: on the path, at the head, or anywhere inside a neutral head). **drop** ends the borrows inside a dropped value and fails on a live loan in a dropped owned value.

Figure 3 gives the rules for borrows. They are Aeneas’s, simplified by treating loans as variables.

Ending a borrow $[\text{End}]$ replaces it by $\perp$ and substitutes its content for its loan. There is no side condition: if the content itself contains loans (because something has reborrowed part of it), they travel with it, and the reborrows remain valid. Ending is allowed at any time; the machine does it lazily, when a place is accessed. $[\text{Access}]$ makes every access exclusive: before a place is read, borrowed, assigned or matched on, every borrow whose loan lies on the path to it or inside its content is ended. A borrower that has been ended holds $\perp$, and any later use of it is an error. This is the whole of the borrow checker: exclusivity of mutable borrows is not a separate analysis but a consequence of running the program.

Reading a place copies its content if the content is borrow-free $[\text{Read}]$ and moves it out otherwise $[\text{Move}]$. Data is therefore copied, and borrows, which must stay unique, are moved; reborrowing is explicit ($\&*x$). A runtime system may implement a last-use copy as a move; an affine usage discipline outside the core licenses this, so that **AddM** performs no allocation. Inside types copying is essential: **Id Nat (Add(x, x)) x** must at least be a well-formed statement, and so must $x + x = 2 \cdot x$.

4.2 Calls and matches

A call evaluates its arguments left to right, each into a temporary of the current frame, so that $[\text{Access}]$ can see and end them: **f(&x, &x)** ends the first borrow when the second is taken, and the call then fails reading $\perp$. It then pushes a frame binding the parameters, runs the body, and pops the frame, dropping its bindings. Popping a frame that still lends out one of its owned places is an error: something would outlive the place it borrows.

Types and proofs are erased at runtime, and the machine treats them accordingly: an erased term is evaluated on a *private copy* of the environment, argument evaluation included, and the copy is then discarded. Which terms are erased is read from syntax and declarations, never from a normal form (Section 6): a call is erased when its callee’s codomain *term* is a sort or has declared sort **Prop**, and any other term when it stands in a type position or its declared type has sort **Prop**. A proof may use mutation freely on its copy, but it cannot affect the real environment, so running a proof and skipping it are indistinguishable and the machine skips it, returning $*$. A call whose head is itself a neutral (an abstract function, or a sealed program) is stuck at once.

match $p \{ Z \Rightarrow t \mid S \ y \Rightarrow u \}$ inspects the head of p ’s content. Z selects the first arm, and $S \ v$ selects the second with y standing for the sub-place $p.1$. Through a borrow, **match** $*x$ leaves the successor in the borrowed content, so $\&y$ then reborrows the predecessor field in place: $x \mapsto \text{borrow}_0 (S \ \text{loan}_1)$ and the new borrow is $\text{borrow}_1 \ v$. If the head is a neutral, the run is stuck.

4.3 Closing off

When the body of a call is stuck, the call itself is stuck, and so is everything waiting for it. A pure type theory would return the stuck call as a neutral term. An effectful call cannot simply be returned: part of what it produces is the final content of the places it borrowed, and the caller’s environment needs those contents before it can continue. We *close off* the call instead.

the body of $f(w_1, \dots, w_n)$ is stuck, $w_i = \text{borrow}_{\ell_i} u_i$ for $i \in I$, u_i loan-free

$$L := \text{let } c_i = u_i \quad (i \in I) \quad C := f(a_1, \dots, a_n), \quad a_i = \begin{cases} \&c_i & \text{if } i \in I \\ w_i & \text{otherwise} \end{cases}$$

result type	result	each loan_{ℓ_i} becomes
Unit	$()$	$[L; C; c_i]$
borrow-free D	$[L; C]$	$[L; C; c_i]$
$\&T$	$\text{borrow}_k [L; \text{let } r = C; *r]$	$[L; \text{let } r = C; *r := \text{loan}_k; c_i]$

Figure 4: [Close]. The partial run of the body is discarded; the borrows borrow_{ℓ_i} are consumed; k is fresh.

Figure 4 gives the rule. Take the call at the point where its arguments have been evaluated. For each borrow argument $\text{borrow}_{\ell_i} u_i$, the canonical program $L; C$ recreates the situation the call was in, but with the borrowed contents owned by fresh local places c_i : it moves each u_i into c_i and calls f on $\&c_i$. This program is closed. Running it and reading c_i afterwards computes exactly what the call would have left behind the borrow, so each loan is filled with the sealed program $[L; C; c_i]$. The call returns the sealed program $[L; C]$, or $()$ if its result type is **Unit**.

Sealed programs are the backward functions of Aeneas, written in the source language. For $\text{AddM}(\text{borrow}_\sigma \sigma, y)$, the loan receives $[\text{let } c = \sigma; \text{AddM}(\&c, y); c]$, which is the body of $\text{Add}(\sigma, y)$. Where Aeneas’s translation would emit `add_m_back` σ y , Ochr writes the program that defines it. Nothing about the call is lost: when σ is later refined, the sealed program runs again.

Returning a borrow. If f returns a borrow, the contents of the places it borrowed are not determined when the call returns: they depend on what the caller will write through the returned borrow before it ends. The rule therefore returns a borrow of a fresh k , whose content is the sealed program for the returned borrow’s initial content, and fills each loan_{ℓ_i} with a sealed program that has a *hole*: it writes loan_k through the returned borrow before reading c_i . Because loans are variables, the hole is filled by the ordinary [End] rule: when the caller drops the returned borrow with final content w , w is substituted for loan_k . This records only the *final* value written through the returned borrow, which is all that the backward function of Aeneas takes as input, and the hole is the neutral form of an Aeneas region abstraction and of a RustHorn prophecy [19]. If f has several borrow arguments, the hole occurs in each of their sealed programs, and one [End] fills them all.

The precondition that each u_i is loan-free is guaranteed by [Access], which ended those loans when the argument was read or borrowed. Without it, a live loan would be copied into a sealed program and survive the borrow that binds it.

4.4 Sealed programs and refinement

Values are kept in normal form. The normal form of a sealed program $[L; C; K]$ is obtained by running it from the empty environment, with one restriction: its *head call* C may not itself be closed off (calls made inside C ’s body may). This is the analogue of the guard on fixpoint unfolding in the calculus of inductive constructions. Without it, a sealed program would re-close into itself forever; with it, a sealed program either runs to a value or stays exactly as it is. A loan whose borrow lives outside the run (a hole whose returned borrow is still live) is *inert*: the run treats it like an abstract value. A borrow error during normalisation is a type error at the point that triggered it. When a case split has generalised a sealed program to an abstract value (Section 6), every later derivation of the same closed program normalises to that value.

A *refinement* substitutes a constructor pattern for an abstract value, $\sigma := Z$ or $\sigma := S \sigma'$, when a case split learns it (Section 6). Substitution into a sealed program is followed by normalisation, which runs the sealed program again. For $N(\sigma) = [\text{let } c = \sigma; \text{AddM}(\&c, \emptyset); c]$ and $\sigma := S \sigma'$, the head call AddM now unfolds, matches the successor, reborrows the predecessor, and makes the inner recursive call, which is stuck on σ' and is closed off; so $N(S \sigma') = S N(\sigma')$. Normal forms of this shape are what makes the induction of Section 2 go through. The key property is that refinement commutes with closing off: refining the arguments of a closed-off call and re-normalising gives the same normal form as closing off the call on the refined arguments (Section 7).

Stuck blocks. A stuck `match` that is not the body of a call is closed off in the same way, as the body of a call to an anonymous function of its free places. Pattern variables are first resolved to the sub-places they denote, and the function takes the resulting maximal places the way Rust infers closure captures: a place the match moves out of, in any arm, is moved in; a place it writes or borrows is passed by borrow; a place it only reads is copied. The function’s result type is the match’s type, which must agree across the arms or be annotated (`let x : A = match ...`). Only a match whose arms have been type-checked is closed off, including a match inside a type, and a stuck block is erased exactly when each of its arms is a declared proof. This covers stuck matches inside types and, during checking, a stuck match that is followed by more code (Section 6).

5 Observational equality

$\text{Id } A \ t \ u$ states that the computations t and u , both of result type A , are indistinguishable from where the statement is made: run from the current environment, they return equal values and leave equal contents in every place either of them may write. This section defines that notion precisely and shows that it reduces to ordinary propositional equality.

5.1 Footprints and observations

A computation affects the world only through the places it names. The places a term may **write** are read off its syntax: they are the free places that appear under $\&__$ or on the left of $:=$, and the borrow variables it mentions, since a borrow grants write access to someone else’s place. Each of these is traced back to the owned place that will receive the write. For a place rooted at an owned variable x , that is x . For a borrow variable holding $\text{borrow}_\ell v$, it is the set of *owners* of t :

$$\text{owners}(\ell) = \bigcup_{\text{occurrences of loan}_\ell} \begin{cases} \text{owners}(m) & \text{if the occurrence lies inside the content of borrow}_m \\ \{x\} & \text{if it lies inside the owned binding } x \end{cases}$$

Following loans outward through enclosing borrows is what connects a reborrow with the place it ultimately writes. In the successor case of `AddMZero`, the argument `borrow1 σ'` has its loan inside the content of x ’s `borrow0`, whose loan is in the generic caller’s c ; so the owner of the recursive call’s argument is c , the same place the goal observes. The owners form a set because a hole left by a returned borrow can occur in several sealed programs at once. Observing only one of them would be unsound: a call that returns a borrow into one of two arguments could then be proved to leave the other unchanged whichever it picked.

The *footprint* $\mathbf{W}(t, u)$ is the union of the owners of the places t or u may write. The places are read off the syntax, after resolving pattern variables to the sub-places they denote, so the footprint does not depend on unrelated places in the environment; their owners are found by following loans, so a later refinement can shrink an owner set when it decides where a hole points, and the component it removes is one both sides agree on.

The *observation* of t at Ω on footprint $\mathbf{W}$ runs t on a private copy of Ω , ends every borrow that remains, and reads off the result and the contents of the footprint:

$$\llbracket t \rrbracket_\Omega^W = (v, \Omega'(k)_{k \in W}) \quad \text{where } \langle \Omega, t \rangle \Downarrow \langle \Omega'', v \rangle \text{ and } \Omega'' \overset{*}{\rightsquigarrow} \Omega' \text{ with no borrows left}$$

If t is itself stuck outside any call, it is first closed off as a stuck block, so an observation always exists. Ending every remaining borrow makes the observation independent of which borrows happened to be live when t finished; that the order in which they are ended does not matter is a theorem (Section 7).

5.2 Id is equality of observations

$$\text{Id } A \ t \ u \equiv \text{Eq } (A \times T_W) \llbracket t \rrbracket_\Omega^W \llbracket u \rrbracket_\Omega^W \quad W = W(t, u), \text{ both sides run from } \Omega \text{ on independent copies}$$

$$\text{Eq } (A \times B)(a, b)(a', b') \equiv \text{Eq } A \ a \ a' \wedge \text{Eq } B \ b \ b' \quad \text{Eq } A \ a \ b \equiv \top \quad \text{if } a \equiv b \quad \top \wedge P \equiv P \equiv P \wedge \top$$

Figure 5: Id computes to Eq between observations (T_W is the product of the types of the places in $\mathbf{W}$; for empty $\mathbf{W}$ the pair is just A), and Eq computes on pairs and on reflexive instances. `refl` : τ .

Figure 5 is the whole of **Id**. Both sides run from the same environment, each on its own copy; sharing one copy and running the sides in sequence would let the first side’s effects change what the second observes. The result type **A** must be borrow-free: a borrow has no meaning once the computation that produced it has finished.

The equality **Eq** on the right is Lean’s, living in the proof-irrelevant universe **Prop**. We add three conversion rules in the style of observational type theory [5, 23]: equality at a product is a conjunction, a reflexive equation is $\top$, and τ is a unit for $\wedge$. Their only role is to strip away the parts of an observation that no side changed, so that **Id Unit (AddM(x, 0)) ()** computes to the single interesting equation

$$\text{Eq (Unit} \times \text{Nat)}((), N(\sigma))(\(), \sigma) \equiv \top \wedge \text{Eq Nat } N(\sigma) \sigma \equiv \text{Eq Nat } N(\sigma) \sigma.$$

Each rule identifies two propositions with the same truth value, so all three hold in the proof-irrelevant set model of Lean’s type theory [8]. Accordingly **refl** proves τ , and by conversion every reflexive equation. Transport **J(A, a, b, P, h, t)** takes its endpoints explicitly, since a reflexive **Eq A a a** computes to $\top$ and no longer records them.

Three consequences are worth drawing out.

Effects are outputs. The observation turns every write to a place in the footprint into an extra component of the result. This is the sense in which **Ochr** has no pure/impure divide: a computation’s effects are part of its value as seen by **Id**, and the programmer never declares them.

Owned locals are observed. The footprint includes places the current function owns, not only the places behind its borrow parameters. This is forced. A context such as $(\square; x)$, which reads x afterwards, distinguishes $x := 6$ from $()$; if **Id** ignored owned locals it would equate them, and transport along that equation would prove **Eq Nat 6 5**.

Unobserved moves are harmless. Reading a borrow variable moves it, so the two sides of an **Id** may consume different borrow variables. Borrow variables are never in the footprint (their owners are), and ending the remaining borrows puts every borrowed content back in its owner, so what one side consumed and the other did not makes no difference to the comparison.

6 Typing

Typing is the machine run on abstract inputs, with case splitting where the checked program itself inspects an abstract value. The judgement $\Omega \vdash t \Downarrow v : A \multimap \Omega'$ says that from Ω the term t runs to the value v of type A , leaving Ω' . It returns the value, not just the type, because later types may depend on it: after **let z = (let y = &x; *y := 2; x)**, the type **Id Nat z 2** holds by **refl** only because the checker knows that z is 2. The rules for the basic forms are the evident ones, each the corresponding machine rule annotated with types, and we omit them. Figure 6 gives the rules that are particular to **Ochr**.

[**Call-type**] At the point where the arguments of $f(\bar{a})$ have been evaluated, with $f : \Pi(\bar{x} : \bar{A}) . B$: push a frame binding each x_i to a_i 's value, evaluate B there on a private copy of the environment, and pop. The result is the type of the call. Each a_i must have type A_i , evaluated with the earlier parameters bound.

[**Def**] $\text{fix } f (\bar{x} : \bar{A}) : B \text{ by } x_j := b$ is checked at its *generic call*: from an environment with a fresh owned place $c_i \mapsto \sigma_i$ for each borrow parameter $x_i : \&T_i$, the call $f(\bar{a})$ with $a_i = \&c_i$ for borrow parameters, $a_i = *$ for proof parameters, and $a_i = \sigma_i$ otherwise; parameter types are evaluated left to right, with the earlier parameters bound, and stored with their bindings. The goal is the [Call-type] of this call. The body runs in the pushed frame, which is then dropped; its result's type must be convertible to the goal, as refined by the splits on the way.

[**Split**] When the checked program matches on a place whose content has an abstract head σ , each arm is checked with the refinement $\sigma := Z$, respectively $\sigma := S \sigma'$ for fresh σ' , applied to the environment, the goal and every stored type. If the head is a sealed program, every occurrence of it is first replaced by a fresh σ . If the match is followed by more code, that code is checked once, from the state in which the match has been closed off as a stuck block.

[**Rec**] In the body of $\text{fix } f \dots \text{by } x_j$, including inside nested functions and block arms, f occurs only as the head of a call, and each recursive call passes in position j a value, or a borrow of a value, that is a strict subterm of x_j 's entry value σ_j as refined so far.

Figure 6: The typing rules particular to Ochr.

6.1 One rule for goals and induction hypotheses

A definition is checked by running it at its most general call [Def]. For $\text{AddMZero}(x : \&\text{Nat})$ that is $\text{AddMZero}(\&c)$ from $\{ c \mapsto \sigma \}$: the parameter becomes a borrow of a fresh owned place, which stands for whatever the real caller lends. The goal is then computed by [Call-type], the rule that types every call.

The same rule types the recursive call in the successor case, at a different environment. There the argument $\&p$ borrows the predecessor field of c 's content, so evaluating the callee's statement at the call site observes c through the successor that surrounds the field. This is how the induction hypothesis arrives already wrapped in S . The generic call and the recursive call are two instances of one rule, which is why the goal and the hypothesis meet without a congruence step. That the callee's statement, proved at its generic call, remains true at every call site is the frame property of Section 7: a call affects only what it is passed, so the caller's observation is the callee's, placed in a context.

6.2 Types are formed once

A type is evaluated once, when it is formed, on a private copy of the environment, and becomes a closed statement about values. Later mutation cannot change it. This includes the codomain of a Π -type, which is a closure over the values of the variables it mentions; only its parameters are bound later, at each call. The goal of a definition is formed at its generic call, before the body runs, so a body that mutates its argument and then proves something about the new contents does not prove the goal: $\lambda(x : \&\text{Nat}) . (*x := 5; \text{refl})$ does not have type $\Pi(x : \&\text{Nat}) . \text{Id Nat } (*x) 5$.

6.3 Case splitting and joins

A *match* in the checked program on an abstract value cannot pick an arm, so each arm is checked with the abstract value refined [Split]. What follows the match is checked once, from a state in which the match itself has been closed off as a stuck block. The closed-off match is precise: its sealed programs contain the match, and they reduce as soon as a later split decides it. There is no join of environments and no loss of information, and a borrow whose origin depends on the branch is handled like any other returned borrow. AddToOne of Section 2 is accepted, and so is any function whose body is split into a helper at a match: a match and a call to a function containing it are treated alike. Errors in the code after a match are reported once, at the line where they occur, not once per arm.

6.4 Erased terms leave no trace

Types and proofs are erased at runtime. The machine mirrors this exactly: it evaluates an erased term on a private copy of the environment, argument evaluation included, and discards the copy. A term is erased when it stands in a type position, when its declared type has sort **Prop**, or when it is a call to a function whose codomain term is a sort or has declared sort **Prop**; the decision never looks at a normal form. Two things follow. A formed type is a closed statement about values, as described above. And a proof can have no effect on the program around it: it may mutate places it creates itself, on its own copy. Running a proof and skipping it are therefore indistinguishable, and the machine skips proofs, which is what the compiled program does too.

Erased terms are also checked not to write, borrow or move a place that outlives them, except by handing it to another erased call. With the private copy this is redundant for a correctly classified term, and it is there as a partial fail-safe: if the two evaluation paths disagreed about whether an inline term is erased, the path that erases it would reject it rather than silently discard its effects. For calls and stuck blocks the fail-safe does not reach inside the callee’s body, and soundness rests on the classification being syntactic (Section 8). This is one principle, not a restriction on programs. **AddMZero**’s successor case borrows the field **p** to pass it to the induction hypothesis; the borrow happens on the private copy, and the real environment is untouched. Nothing in a program is marked pure, and any program may appear in a statement.

6.5 Why each condition is there

Each side condition was added in response to a concrete closed proof of false, or an accepted program that goes wrong, found while designing the calculus; each is a regression test in the implementation, and switching it off lets its counterexamples back in (Section 8). Most of them guard one invariant: a statement is evaluated once through closing off, at a definition’s generic call, and again directly at each instance, and **[Call-type]** and **[Split]** equate the two, so every decision the two paths must agree on is made from syntax (Lemma 7).

Condition	What goes wrong without it
Π -types capture values	$h : \Pi(_ : \text{Unit}). \text{Id Nat } x \ Z$ proved when x was Z is re-read after $x := S \ x$, proving $\text{Id Nat } (S \ Z) \ Z$.
Recursion on entry values; f only as a call head; no f without by	$*x := S \ *x$; $\text{match } *x \{ S \ p \Rightarrow f(\&p) \}$ recurses on the original value; passing f to a helper or calling it in a by -less body avoids the check; each proves $\text{Eq Nat } 0 \ 1$.
Erased terms run on a private copy, and may not affect outside places	Proof irrelevance identifies $\lambda x. \ * \text{ with } \lambda x. \ (*x := 7; \ *)$; skipping only proof <i>calls</i> lets a closed-off proof block and the same block run inline disagree.
Erasure read from syntax, never from normal forms; universes not cumulative	A call $W(\&c, n) : U(n)$ with $U(n) := \text{match } n \{ Z \Rightarrow \text{Prop} \mid \dots \}$ runs at the generic n but is erased at $n = Z$; the same for a local function whose codomain mentions captured values, and for a type-valued match erased only when closed off.
Functions compared by their observation	Comparing results alone identifies $\lambda x. \ (*x := S \ Z)$ with $\lambda x. \ ()$; comparing borrow-returning functions without writing through the result identifies $\lambda(x, y). \ x$ with $\lambda(x, y). \ y$.
Pattern variables are places	A stuck block that writes through p in $\text{match } *x \{ S \ p \Rightarrow p := Z \}$ captures $*x$ by copy.
All owners observed	A borrow returned into one of two arguments leaves its hole in both; observing one proves $\text{Eq Nat } 0 \ 1$.
Exclusive access; loan-free closing off; matches end loans in neutral heads	A live loan copied into a sealed program, or a hole generalised away by a split, lets an accepted program write through an ended borrow.
Generalisations are global; fresh names never reused	A generalisation made while forming a type is lost with its private copy, and its name is reissued for a different computation.
Strict positivity	$\text{inductive Bad} := \text{Mk}(f : \Pi(x : \text{Bad}). \text{Empty})$ proves $\text{Eq Nat } 0 \ 1$ through a proof that is never run.

Figure 7: The side conditions of Ochr and the counterexamples that forced them.

7 Metatheory

We show that *Ochr* is consistent and that the checker’s symbolic evaluation computes what programs do. Following Aeneas [12], a computation is read as a pure function from the current contents of its borrows to its result and their final contents; types get a set-theoretic interpretation after Carneiro [8]. We give precise statements and the hardest case of each argument; Section 7.6 says what is mechanised.

7.1 The model

What consistency is relative to. Computations translate into CIC_L , the intensional type theory of Lean 4 [20], with an impredicative, definitionally proof-irrelevant **Prop** and **propext**, and every machine step becomes a conversion there (Theorem 3). Types need more. *Ochr*’s conversion identifies propositions that CIC only proves equivalent, through the **Eq** rules (Section 5) and through [Call-type], which identifies a call site’s type with the one proved at the generic call (Theorem 4). So no translation into intensional CIC preserves conversion. Equality reflection would, but Carneiro’s set model relies on unique typing, which reflection breaks, and eliminating reflection [26] is established only for sources without an impredicative, proof-irrelevant **Prop**. We therefore interpret *Ochr*’s types directly in sets, following Carneiro’s construction and extending his interpretation of the CIC_L translation of computations. Carneiro needs unique typing to decide whether a λ or Π denotes a proof or a set-theoretic function; in *Ochr* that decision is syntactic, because definitions declare their codomains, erasure is read from declared sorts and syntactic positions, and universes are not cumulative (Section 6). The interpretation is therefore defined by recursion on derivations, and consistency is relative to ZFC with one inaccessible cardinal per universe level. It exists on paper only.

Types. Data and propositions denote themselves: **Nat** denotes $\mathbb{N}$ and **Eq A a b** denotes $[a^\dagger = b^\dagger] \subseteq \{\bullet\}$. For a function type with borrow parameters $\mathbf{x}_i : \&\mathbf{T}_i$ at positions $i \in I$, let $D_i = T_i^\dagger$ for $i \in I$, $D_j = A_j^\dagger$ otherwise, and $\text{Fin} = \prod_{i \in I} T_i^\dagger$, the final contents of the borrowed places:

$$(\Pi(\bar{x} : \bar{A}).B)^\dagger = R_{\Pi(\bar{x}:\bar{A}).B} \subseteq \Pi(\bar{d} : \bar{D}).\text{Out}_B(\bar{d}), \quad \text{Out}_B(\bar{d}) = \begin{cases} B^\dagger(\bar{d}) & \text{if } B \text{ is a proposition,} \\ B^\dagger(\bar{d}) \times \text{Fin} & \text{if } B \text{ is borrow-free data,} \\ T^\dagger \times (T^\dagger \rightarrow \text{Fin}) & \text{if } B = \&T. \end{cases}$$

Here $B^\dagger(\bar{d})$ is B at the generic call of [Def]. We write fwd_F and back_F for the two components of $F(\bar{d})$. For a returned borrow, $\text{back}_F(\bar{d})$ is Aeneas’s backward function: it maps the value the caller eventually leaves in the borrow to the final contents. A proposition has no Fin component: proofs run on a private copy (Section 6), so they are not state transformers, and proof irrelevance is sound.

Definition 1 (injectivity relation). $R_A \subseteq A^\dagger$ is defined by induction on A . For base types, propositions and universes $R_A = A^\dagger$; R is componentwise on pairs, and $R_{\&T} = R_T$. For a function type, $F \in R_{\Pi(\bar{x}:\bar{A}).B}$ iff, for all $\bar{d}$ with $d_i \in R_{T_i}$ for $i \in I$ and $d_j \in R_{A_j}$ otherwise, $\text{fwd}_F(\bar{d}) \in R_B$, and, if $B = \&T$, $\text{back}_F(\bar{d})$ is injective: equal tuples of final contents come only from equal final values of the returned borrow.

R is forced. By [Call-type], *Ochr* proves that every *abstract h* : $\Pi(\mathbf{x} : \&\mathbf{Nat}). \&\mathbf{Nat}$ has an injective backward function, which is false of arbitrary set-theoretic functions (Lemma 6). Consequently an opaque definition of a borrow-returning type is an injectivity assumption, and binding it to an arbitrary external function is unsound.

Terms. Abstract values become variables. A loan ℓ becomes a variable h_ℓ , a *hole* for the final content of its borrow. The *view* $\Omega^\dagger$ reads a borrow as its current content and a loan as its hole; the *resolution* ρ_Ω substitutes each hole by its borrow’s content, that is, it ends every borrow. On the *shapes* of one run (which bindings exist, and where borrows and loans sit), a term translates to a state-passing function on views (Figure 8). A sealed program denotes what it computes, $[t]^\dagger = \text{Run}(t)$, the result of $t^\dagger$ run from the empty view; a definition is a well-founded recursion on the entry value of its decreasing parameter; and **Id** is the equation between the tuples of result and resolved footprint. A symbolic run and a concrete run of one term can pass through different shapes (Section 7.4), so the translation is defined per run, and runs are related through their resolutions.

$$\begin{aligned}
\text{End}_\ell &: s \mapsto (s \setminus \text{holder}(\ell))[h_\ell := s.\text{holder}(\ell)] \\
\text{match } p\{\dots\} &: s \mapsto \text{natCase } (s.p) \left(t_Z^\dagger(s[p := 0]) \right) \left(\lambda\sigma'. t_S^\dagger(s[p := \text{succ } \sigma']) \right) \\
f(\bar{a}), B \text{ data} &: s \mapsto \text{let } (r, \bar{\varphi}) = f^\dagger(\bar{u}, \bar{w}) \text{ in } \left(r, (s \setminus \bar{a})[h_{\ell_i} := \varphi_i] \right) \\
f(\bar{a}), B = \&T &: s \mapsto \text{let } (c, \beta) = f^\dagger(\bar{u}, \bar{w}) \text{ in } \left(c, (s \setminus \bar{a})[h_{\ell_i} := \beta_i(h_k)] \right)
\end{aligned}$$

Figure 8: Key clauses of the translation. $\bar{u}$ are the contents of the borrow arguments, whose loans are ℓ_i , and $\bar{w}$ the other arguments; k is fresh. Ending a borrow is a substitution, and a call is one application, with Aeneas’s region abstraction as the substitution $h_{\ell_i} := \beta_i(h_k)$. An erased term leaves the view unchanged.

An environment is *well formed* when each borrow is held once, every loan is bound by a held borrow or lies inside a sealed program (the only place a loan may occur twice), loans nest in borrows acyclically, and every value read, moved or passed is loan-free.

Lemma 2 (invariance; termination). Machine steps preserve well-formedness. For programs accepted by [Rec], every concrete run terminates.

Concrete runs never evaluate types or proofs, so termination reduces to the recursion [Rec] enforces. Whether type checking is decidable is open: types may combine large elimination, **J** with type-valued motives and an impredicative, proof-irrelevant **Prop**, which Abel and Coquand show can defeat normalisation [1]. Nothing below depends on decidability, since the model interprets derivations.

7.2 Sealed programs are backward functions

Theorem 3 (simulation). If Ω is well formed and $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$, then on that run’s shapes $t^\dagger(\Omega^\dagger) \equiv (v^\dagger, \Omega'^\dagger)$ in CIC_L . The sealed programs that [Close] produces at $f(\bar{a})$, with borrow contents $\bar{u}$ and other arguments $\bar{w}$, satisfy

$$[\text{L}; \text{C}]^\dagger \equiv \text{fwd}_f(\bar{u}, \bar{w}), \quad [\text{L}; \text{C}; c_i]^\dagger \equiv \text{back}_f(\bar{u}, \bar{w})_i, \quad [\text{L}; \text{let } r = \text{C}; *r := \text{loan}_k; c_i]^\dagger \equiv \text{back}_f(\bar{u}, \bar{w})(h_k)_i.$$

The equations are definitional: a sealed program is a name, in source syntax, for an application of a backward function. The routine steps are conversions ([End] is substitution, a match on a constructor is ι , unfolding is $\delta\beta$). The hardest case ends a borrow k whose loan is a hole inside sealed programs. The model substitutes $h_k := w^\dagger$ into $\text{Run}(t)$; the machine substitutes w into t and runs it again. These agree only because [Seal] treats a loan whose borrow lies outside the run as inert: a run that could end k would reach outside the sealed program, and $\text{Run}(t)$ would not be a function of t .

7.3 A call affects only what it is passed

For a call at Ω whose borrow arguments hold the loans $\bar{\ell}$, and a set O of owners of those loans, let $\text{Ctx}_O(\bar{\varphi}) = (\rho_\Omega(o)[h_{\bar{\ell}} := \bar{\varphi}])_{o \in O}$: the owners’ resolved contents, as a function of the final contents $\bar{\varphi}$ of the borrowed places.

Theorem 4 (frame). Let Ω be well formed.

1. *Locality.* If $\Omega = \Omega_1 \uplus \Omega_2$, t ’s free variables are bound in Ω_1 , and every loan in Ω_1 has its borrow in Ω_1 , then t runs from Ω iff it runs from Ω_1 , with the same result, and Ω_2 changes only by filling the loans of the borrows the run ended.
2. *Call effect.* A call runs its body from its own frame alone, then substitutes each borrow argument’s final content for its loan.
3. *Call typing.* Let O be *all* owners of the loans of a call’s borrow arguments. Each **Id**-atom of the callee’s codomain, observed at the call site, is the one observed at the generic call mapped through Ctx_O ; so when Ctx_O is injective, the two types denote the same set.

Lemma 5 (owners and injectivity). If every function value in Ω lies in R and O contains every owner of every occurrence of $\bar{\ell}$, then Ctx_O is injective. The condition on O is necessary.

Lemma 6 (backward functions are injective). If every opaque definition lies in R , so does every definition accepted by [Def] and [Rec].

Part (3) is what [Call-type] rests on, and why an induction hypothesis arrives in its caller's context, as $((), S \sqcap)$ in **AddMZero**: both observations run on private copies, which by (1) change nothing outside the arguments, and resolving at the end plugs the arguments' final contents into their owners. Lemma 5 holds because Ctx_O is built from constructors, from backward maps (injective by R), and from components constant in the hole, which is why an owner carrying the hole must be kept; for **Pick**(n , $\&a$, $\&b$) closed off on an abstract n , $\text{Ctx}_{\{a\}}$ is constant. Lemma 6 is the fundamental lemma of R , proved jointly with Theorem 10; function-typed parameters range over R by definition.

The hardest case is part (2) for a returned borrow. As the frame pops, the parameter's borrow still holds the returned borrow's loan, so the caller's owner receives $K[\text{loan}_r]$, which depends on a value the caller has not yet produced. Its injectivity, $\beta_i(h_r) = K[h_r]$, must hold for every future of r , so it cannot be read off Ω : it needs the induction over definitions, and the fact that no rule discards a hole's final value ([Access] ends the loans in a content before overwriting or dropping it).

7.4 The two paths agree

A statement is evaluated along two paths: at a definition's generic call, where calls and matches on abstract values close off, and directly at each instance. [Call-type] and [Split] identify the results, so any decision taken differently on the two paths is an inconsistency. A *refinement* α substitutes values for abstract values, definable closures for abstract functions, and values for holes; it covers both case splits and instantiation at a call site. $\Omega \downarrow$ re-normalises Ω 's sealed programs, and $\approx$ is equality up to renaming of loans.

Lemma 7 (closing off commutes with refinement). For t at Ω and a refinement α , the following are the same for t at Ω and $t\alpha$ at $\Omega\alpha$: (1) which subterms are erased; (2) which places each closed-off stuck block captures, in which modes, and the places of each footprint; (3) whether each match's arms are well typed; (4) each call's [Close] row.

Each decision is read from syntax or declarations, never from a normal form, and α changes only normal forms: (1) a call is erased iff its callee's codomain *term*, read without evaluating it, is a sort or has declared sort **Prop**, a stuck block is erased exactly when its match is, and universes are not cumulative; (2) pattern variables are resolved to the sub-places they denote before captures and footprints are computed, so a write through p in **match** $*x \{ S \ p \Rightarrow \dots \}$ is a write to $*x$ on both paths; (3) arms are checked under every refinement of the scrutinee, inside types too, and a match still neutral after α stays closed off; (4) the row is read from the declared result type. Deciding any of these from normal forms admits a closed proof of **Eq Nat Z (S Z)** (Section 6).

Theorem 8 (schedule independence and naturality). Let Ω be well formed.

1. Ending two borrows in either order gives the same resolution. If a run of t succeeds while ending some borrows earlier than the lazy strategy would, the lazy run succeeds with the same resolution and the same borrow-free result. Hence observations do not depend on when borrows end, and **Id** is well defined.
2. If $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$ symbolically and α refines Ω , then $((\Omega'\alpha) \downarrow, (v\alpha) \downarrow)$ is, up to $\approx$, the result of a run of $t\alpha$ from $(\Omega\alpha) \downarrow$ that ends some borrows early. With (1), for every footprint W fixed on both sides, $(\llbracket t \rrbracket_{\Omega}^W \alpha) \Downarrow = \llbracket t\alpha \rrbracket_{\Omega\alpha}^W$.

Corollary 9 (adequacy). If α is ground and instantiates every opaque definition with a definable function, the lazy run of $t\alpha$ is concrete, terminates, and computes the instantiated symbolic observation: every **Id** the checker computes on abstract inputs holds on every concrete input.

Every step except [Close] commutes with α on the nose, by Lemma 7. At a [Close], re-normalising the sealed programs runs the call's body alone on the refined arguments (Theorem 4), as the direct run does, and [Seal] unfolds only the head call, so inner calls close off where the direct run's do. The hardest case is a returned borrow. The concrete run puts its loan into the one owner it points into, but the symbolic hole sits in the fill of every borrow argument it might point into, so the symbolic run ends the borrow when *any* possible owner is accessed. That is an extra [End], which (1) absorbs. The owners of a footprint are therefore not syntactic: when α removes a hole from a sealed program, the owner that loses it is unchanged on both sides, and the $\top$ rules strip its component.

The extra [End] is real and costs completeness. With **Pick**(n , x , y) := **match** $n \{ Z \Rightarrow x \mid S _ \Rightarrow y \}$ and n abstract,

```
r := Pick(n, &a, &b); z := b; match n { Z => *r := 5 | S _ => () }
```

runs for every concrete n but is rejected: reading b ends r symbolically, since `Pick`’s hole sits in the fills of both a and b , so r is dead in the Z arm. After $z := b$ the symbolic and concrete states agree only after resolution (machine-checked, Section 7.6), which is why (2) is stated up to early ends; Rust likewise treats r as borrowing both places while it is live.

Naturality and thunkability. With effects in types, stability under substitution can fail in a new way. If erasure were read from normal forms, take $U(n)$ of Section 6, which is `Prop` for every n , and $W(x : \&\text{Nat}, n : \text{Nat}) : U(n) := *x := S\ Z; V(n)$. At the generic n , $U(\sigma)$ is stuck, so $W(\&c, n)$ runs and writes c ; at $n = Z$ it would be erased. Then `Id Nat (let c = Z; W(&c, n); c) (S Z)`, proved by `refl` generically, states `Eq Nat Z (S Z)` at Z . This is exactly Pédrot and Tabareau’s “desynchronisation between effects performed in the term and effects performed in the type” [24], which Lemma 7 rules out. They show that a computation is thunkable iff it is natural, that is, running then restricting equals restricting then running [24:Prop. 18]; Lemma 7 and Theorem 8 are the naturality Ochr needs. We claim a correspondence of invariants, not an embedding into their calculus.

7.5 Soundness and consistency

Theorem 10 (soundness of the model). For a program accepted by [Def] and [Rec], and every valuation of the abstract values: types of sort s denote elements of s ’s universe; convertible types denote the same set; if $\Omega \vdash t \Downarrow v : A \dashv \Omega'$, then on that run’s shapes t denotes a function from views of Ω to elements of A ’s denotation and views of Ω' ; and every definition lies in R .

The proof is by induction on derivations, jointly with Lemma 6; most cases read the translation backwards. [Split] on a sealed program generalises it first, which is well typed because stored types are normal forms, so every dependency on the program is syntactic, and sound only because matching first ends every loan inside a neutral head. Closures are convertible when their generic calls have the same observation, which determines their denotation; comparing results alone would identify $\lambda x. (*x := S\ Z)$ with $\lambda x. ()$. The `Eq` rules hold because convertible terms denote equal elements and propositions with the same truth value are equal. [Rec] recurses on data, never on proofs, so no subterm check passes through a cast along a `propext` equality. The hardest case is [Call-type], which combines Theorem 4 (3), Lemma 5, Lemma 6 and Theorem 8, and needs Π -types to capture their free variables when formed, so that a call site’s type is an instance of the proved one.

Corollary 11 (consistency). Relative to ZFC with one inaccessible per universe level, no closed term has type `Eq Nat Z (S Z)`, and every closed proposition with a closed proof holds in the set model.

7.6 What is mechanised

A Lean 4 development mechanises the effectful layer for a first-order fragment, following an earlier version of the rules: natural numbers, unit, pairs, places, borrows, matching, recursive definitions (including borrow-returning ones), opaque definitions and erased proofs, with no closures, types or stuck blocks. The machine is a clocked big-step interpreter. Proved with no `sorry`, using only Lean’s standard axioms: parts (1) and (2) of Theorem 4, as equations at every amount of fuel; that the sealed program [Close] writes into a borrowed place normalises to the content the call leaves there (the machine-level form of the second equation of Theorem 3), assuming the call’s results are loan-free, which should follow from Lemma 2; that two borrow endings commute (the first claim of Theorem 8 (1), without re-normalisation); and that the machine is deterministic, monotone in fuel, and equivariant under renaming of loans. Executable tests check the other [Close] equations on ground inputs, owner sets, erasure and the `Pick` example. The invariance part of Lemma 2 is stated with its proof still a `sorry`, and the remaining [Close] equations are in progress. Not started: the translation into CIC_L , the rest of Theorem 8, Lemma 7, Lemma 5 and Lemma 6. The set-theoretic interpretation, and so Theorem 10 and Corollary 11, exists on paper only.

8 Implementation

We implemented the calculus as an executable reference checker in Lean 4, about 3,300 lines with no dependencies, plus about 1,100 lines of examples and tests. Its purpose is not performance but fidelity: a hand derivation can be charitable where the rules are silent, and a program cannot. Every example in this paper is checked by it, written in a concrete syntax close to the one used here:

```
ochr E1 {
  def AddM (x : &Nat) (y : Nat) : Unit by x :=
    match *x { Z => *x := y | S p => AddM(&p, y) }

  def AddMZero (x : &Nat) : Id Unit (AddM(x, 0)) () by x :=
    match *x { Z => refl | S p => AddMZero(&p) }

  reject def WriteThenRefl (x : &Nat) : Id Nat (*x) 5 := *x := 5; refl
}
```

A `def` is expected to be accepted and a `reject def` to be rejected, and each verdict is an assertion checked during the build, so a build succeeds only if every verdict is as expected. Each test file also asserts how many verdicts it contains, which guards against a test file that silently checks nothing.

Structure. The machine, closing off, sealed-program normalisation, splitting, closing off stuck blocks, observation, `Id`, `[Call-type]`, `[Def]` and `[Rec]` form one mutually recursive block of about a thousand lines. Terms use de Bruijn indices, and the variables introduced by `[Close]` have fixed names, so “same normal form” is structural equality. The implementation follows the rules closely enough that its trace of `AddMZero` reproduces the derivation of Section 2 symbol for symbol, including the successor that the environment supplies to the induction hypothesis.

Test suite. The suite contains every example of this paper: `AddM`, `AddMZero`, `Add` and `AddZero`; the returned-borrow examples `TailM`, `AddM'` and `AddMEq`; branching with borrows whose origin depends on the branch; opaque function arguments; and the programs of Section 2 that pass a proof about the current, mutated state. It also contains every program that must be rejected, and in particular every false proof found while designing the calculus, each of which is a regression test. The suite also contains user-declared inductive types: in-place list append and its returned-borrow variant, and the binary-search-tree insertion of Section 2 with its size theorem. All 239 verdicts are as expected. The compiled checker decides the whole suite in about 11 ms, the heaviest declaration (the size theorem) in under a millisecond; a clean build takes under half a minute, most of it spent re-running the suite for the thirty rows of the counterfactual ledger.

Every side condition is load-bearing. Each non-standard rule has a switch that turns it off, and the build also asserts a *counterfactual ledger*: with one rule switched off, exactly the tests that rule is responsible for change their verdict, and no others. Without entry-value recursion (Section 6), the non-terminating “proofs” of Section 6 are accepted. Without exclusive access, a well-typed program writes through an ended borrow. Without running erased terms on a private copy, a proof that writes changes the verdict on programs around it. And without the rule that a recursive function occurs only as the head of a call, a function that passes itself to a helper proves `Eq Nat 0 1`. Switching off owner sets (Section 5) admits a closed proof of `Eq Nat 0 1` built from a function that returns a borrow into one of two arguments and an annotated `match` whose type is formed while the hole is in both owners; with owner sets, the arm that picks the second owner must prove a false equation and is rejected.

What running the rules found. Writing the checker exposed gaps in earlier versions of the rules, each now fixed: the recursive-function-as-a-value loophole above; an argument evaluated earlier in the same call being invalidated by a later one (`f(&x, x)` ends the first borrow while reading the second argument); recursive calls inside nested closures, which must be checked against the entry values of the enclosing function; and, with trees, the need to apply a split on a sealed program to re-derivations of it as well. Every closed false proof found by the reviews and adversarial rounds summarised in Figure 7, including several that an earlier version of the checker accepted, is a regression test, and every row of the counterfactual ledger flips at least one of them.

9 Related work

Functional translation of borrows. Aeneas [12] translates safe Rust into a pure functional program in Lean, F* or Coq. Its symbolic semantics of the low-level borrow calculus is the direct ancestor of our machine: we take its values with loans and borrows, its lazy ending of borrows, and its insight that a mutable borrow parameter is a forward computation plus a backward function. Later work gives that symbolic semantics joins and loops and proves it sound as a borrow checker [13]. The difference is where and when the translation happens. Aeneas translates eagerly, whole-program, once, and the user reasons about the translated artifact, where backward functions are named definitions. Ochr never produces a translated program. The same state-passing content appears lazily inside conversion, only as far as a particular equation needs, and only in source syntax: a sealed program is what Aeneas would call an application of a backward function. Aeneas is modular in that it never unfolds a callee; Ochr must unfold, because unfolding is what definitional equality is, and closing off is what keeps the unfolding finite.

Prophecies. RustHorn [19], RustHornBelt [18] and Creusot [9] model a mutable borrow as its current value paired with a *prophecy* $\hat{x}$ of its final value, resolved when the borrow ends. A filled loan in Ochr is a prophecy that has been resolved by the machine, and the hole in a sealed program for a returned borrow is an unresolved one. Prophecies are logic variables introduced at borrow creation; Ochr introduces no variables at all outside binders, which is what keeps concrete evaluation free of symbols (§Section 7).

Specification-based verification of imperative code. Verus [15], Creusot, Prusti [6], Flux [16] and VeriFast [14] verify imperative code against specifications written separately, in pre- and postconditions, refinement types or separation logic, and discharge the obligations with automated provers. Hoare type theory [21] and Ynot [22] internalise the triples in a dependent type theory, as do F*’s separation-logic DSLs Steel and Pulse [10]; F*’s Dijkstra monads [3] internalise weakest preconditions and let `reify` turn a stateful computation into a state-passing function. Relational reasoning about two effectful programs is possible in some of these, notably through F*’s `reify`, which exposes a stateful computation as a state-passing function that can then be compared; the comparison is about the reified pure function. In Ochr the specification of an in-place function is another program, or a theorem about the function itself, equality of computations is a type, and it is decided on the effectful code by conversion where possible.

Effects in dependent type theory. The fire triangle [24] shows that substitution, dependent elimination and observable effects cannot coexist consistently, and ∂ CBPV organises the possible compromises. Closed Ochr programs are not observably effectful in their sense, since ownership confines a closed program’s effects to places it created, so the no-go theorem does not apply to them directly; the danger lies with open terms, where it appears as their desynchronisation between the effects of a term and of its type (Section 1.3). Ochr keeps dependent elimination and restricts what enters a type to values: a computation in a type contributes only its observation, which is *thunkable* in their sense, and ownership gives a syntactic criterion for thunkability, which they note is otherwise hard to capture intensionally. Unlike their call-by-value corner, Ochr retains dependent `let` and large elimination, because typing threads the values computed so far. Ahman’s eMLTT [4] equips computations with definitional equations from an algebraic theory of global state; Ochr’s state is structured, local and borrow-shaped, and its equations come from running the program rather than from an axiomatised theory.

Observational equality. Observational type theory [5, 23] computes the equality type by the structure of the type: equality of pairs is a pair of equalities, equality of functions is pointwise. Ochr’s `Id` between computations is an instance of this idea at the type of observations. We adopt only the rules for pairs and for reflexive equations, which is what lets `Id` compute to the one interesting equation; adding the constructor rules for `Nat` (`Eq Nat (S a) (S b)  $\equiv$  Eq Nat a b`) would be sound in the same model and would remove the remaining congruence steps from pure proofs.

Pure languages with in-place updates. Lean 4 performs destructive updates on values it can prove unshared at runtime [25], and Koka’s fully in-place fragment [17] guarantees statically that a function allocates nothing. Programs in these languages are pure, so their proofs are ordinary, but the programmer writes functional code and cannot express a borrow that is returned to the caller. Linear and quantitative type theories [7] and ATS

[27] track resources in types; Ochr uses the same separation between runtime usage and erased type-level usage as quantitative type theory, where a variable used in a type is not consumed.

Relational reasoning. ReLoC [11] and relational Hoare logics prove two programs equivalent inside a program logic. Ochr’s `Id` is a relational statement about two computations, but it is decided, where possible, by conversion, and it lives in the same type theory as the programs.

10 Discussion

10.1 What generalises directly

Inductive types. The rules are stated for natural numbers, but nothing in them is specific to numbers, and the implementation supports user-declared inductive types (Section 2, Section 8): a constructor with several fields gives several sub-places, `[Split]` refines an abstract value to a constructor applied to fresh abstract values, and `[Rec]` asks for a strict subterm of the entry value. Indexed families are future work.

Opaque definitions. A definition may be declared with a type and no body, or used through an abstract function value. A call to it is stuck at once and closed off, so its effects are recorded as sealed programs mentioning it, and every theorem about its callers remains available. This is how Ochr recovers the modularity of Aeneas’s symbolic execution [12], which never unfolds a callee: the programmer chooses, per definition, whether conversion may look inside. One caveat carries over from the model (Section 7): an opaque definition that returns a borrow is an *assumption* that its backward function is injective, so binding it to an arbitrary external function is unsound in general.

Shared borrows. Immutable borrows are simpler than mutable ones in this setting: a shared borrow never has a final value different from its initial one, so closing off a call that takes one needs no sealed program for it. We omitted them to keep the core small.

10.2 The frontier

Borrows stored in data. The central restriction of the core is that borrow types never occur inside other types. It rules out an iterator that returns a list of borrows into a list, `IterM : &(List Nat) → List (&Nat)`. There, the number of borrows returned is itself unknown on symbolic input, so no fixed set of holes can stand for their final values. The natural generalisation is a hole that stands for a whole data structure of final values, filled when the last borrow in the structure ends, which is the neutral form of an Aeneas region abstraction over a symbolic value with a borrow projector. The same restriction excludes closures that capture borrows.

Loops. A loop is a tail-recursive local function, and its type is its invariant. Closing off a loop that is stuck on an abstract value is closing off that function. We have not designed the surface form, nor checked whether the invariants that arise are the ones programmers expect to write.

Interior mutability and unsafe code. Closing off is sound because a call can affect only what it is passed. Global state, interior mutability or unsafe aliasing would each break this frame property, and with it the rule. They would need to be confined, as Rust confines unsafe code behind safe interfaces, and given sealed-program semantics through their interfaces.

10.3 Costs

Checking cost. Type checking runs programs. On concrete data this is the cost of running them; on symbolic data each stuck call is closed off once, and each refinement re-runs the sealed programs that mention the refined value. Sealed programs are closed and canonical, so their normal forms can be cached and shared, and a refinement only re-runs what mentions it. Our implementation does no caching and decides every example of this paper in about 9 ms in total; we have not measured large programs.

Copying. The calculus copies borrow-free data on reads. A compiled program should move instead, and an affine usage discipline on runtime code, outside the core, licenses implementing each last-use copy as a move.

Types are exempt from that discipline, which is what lets $x + x = 2 \cdot x$ be stated for a type without a copy operation; this is the same separation between runtime usage and erased usage as quantitative type theory [7].

What the programmer sees. Goals and hypotheses are propositions about values, and every value that appears in them is either data, an abstract value named after the parameter it came from, or a sealed program, which is source code. A sealed program can be printed as the user’s own wrapper when one exists (`[let c = σ; AddM(&c, θ); c]` is `Add(x, θ)`). Nothing in a goal mentions loans, borrows, environments or backward functions. We regard this as a design invariant, and it is a useful test for extensions: a feature whose goals cannot be printed as source code is one the programmer would need a new concept to reason about.

11 Conclusion

The usual way to verify an efficient imperative program is to write it twice. We have shown that, for programs whose mutation is disciplined by Rust-style ownership, the second copy can be generated on demand, inside the type checker, as source code: when definitional equality meets a call it cannot run, it closes the call off into sealed programs that own their state, and a later case split runs them again. Combined with an observational equality that compares computations by their results and their writes, this lets theorems about in-place code be stated about that code and proved by the same structural recursion one would use for pure code. Often the in-place proof is the simpler one, because the borrow structure of the environment performs the congruence steps a pure proof must spell out.

The core calculus is small, and every side condition in it is there because a smaller one admitted a proof of false. The main open direction is borrows stored inside data, where the neutral form of a stuck call must describe a whole structure of final values rather than a fixed set of places.

Bibliography

- [1] Andreas Abel and Thierry Coquand. 2020. Failure of Normalization in Impredicative Type Theory with Proof-Irrelevant Propositional Equality. *Logical Methods in Computer Science* 16, 2 (2020).
- [2] Andreas Abel. 2013. Normalization by Evaluation: Dependent Types and Impredicativity. In *Habilitation thesis, LMU Munich*, 2013.
- [3] Danel Ahman, Catalin Hritcu, Kenji Maillard, Guido Martínez, Gordon Plotkin, Jonathan Protzenko, Aseem Rastogi, and Nikhil Swamy. 2017. Dijkstra Monads for Free. In *POPL*, 2017.
- [4] Danel Ahman. 2018. Handling Fibred Algebraic Effects. In *Proc. ACM Program. Lang. (POPL)*, 2018.
- [5] Thorsten Altenkirch, Conor McBride, and Wouter Swierstra. 2007. Observational Equality, Now!. In *PLPV*, 2007.
- [6] Vytautas Astrauskas, Peter Müller, Federico Poli, and Alexander J. Summers. 2019. Leveraging Rust Types for Modular Specification and Verification. In *Proc. ACM Program. Lang. (OOPSLA)*, 2019.
- [7] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *LICS*, 2018.
- [8] Mario Carneiro. 2019. The Type Theory of Lean.
- [9] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *ICFEM*, 2022.
- [10] Aymeric Fromherz, Aseem Rastogi, Nikhil Swamy, Sydney Gibson, Guido Martínez, Denis Merigoux, and Tahina Ramananandro. 2021. Steel: Proof-Oriented Programming in a Dependently Typed Concurrent Separation Logic. In *Proc. ACM Program. Lang. (ICFP)*, 2021.
- [11] Dan Frumin, Robbert Krebbers, and Lars Birkedal. 2018. ReLoC: A Mechanised Relational Logic for Fine-Grained Concurrency. In *LICS*, 2018.
- [12] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust Verification by Functional Translation. In *Proc. ACM Program. Lang. (ICFP)*, 2022. 116:1–116:31.

- [13] Son Ho, Aymeric Fromherz, and Jonathan Protzenko. 2024. Sound Borrow-Checking for Rust via Symbolic Semantics. In *Proc. ACM Program. Lang. (ICFP)*, 2024.
- [14] Bart Jacobs, Jan Smans, Pieter Philippaerts, Frédéric Vogels, Willem Penninckx, and Frank Piessens. 2011. VeriFast: A Powerful, Sound, Predictable, Fast Verifier for C and Java. In *NASA Formal Methods (NFM)*, 2011.
- [15] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. In *Proc. ACM Program. Lang. (OOPSLA)*, 2023.
- [16] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid Types for Rust. In *PLDI*, 2023.
- [17] Anton Lorenzen, Daan Leijen, and Wouter Swierstra. 2023. FP^2 : Fully in-Place Functional Programming. In *Proc. ACM Program. Lang. (ICFP)*, 2023.
- [18] Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code. In *PLDI*, 2022.
- [19] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-based Verification for Rust Programs. In *ESOP*, 2020.
- [20] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *CADE*, 2021.
- [21] Aleksandar Nanevski, Greg Morrisett, and Lars Birkedal. 2006. Polymorphism and Separation in Hoare Type Theory. In *ICFP*, 2006.
- [22] Aleksandar Nanevski, Greg Morrisett, Avraham Shinnar, Paul Govereau, and Lars Birkedal. 2008. Ynot: Dependent Types for Imperative Programs. In *ICFP*, 2008.
- [23] Loïc Pujet and Nicolas Tabareau. 2022. Observational Equality: Now for Good. *Proc. ACM Program. Lang. (POPL)* 6, (2022).
- [24] Pierre-Marie Pédrot and Nicolas Tabareau. 2020. The Fire Triangle: How to Mix Substitution, Dependent Elimination, and Effects. *Proc. ACM Program. Lang. (POPL)* 4, (2020), 58:1–58:28.
- [25] Sebastian Ullrich and Leonardo de Moura. 2019. Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming. In *IFL*, 2019.
- [26] Théo Winterhalter, Matthieu Sozeau, and Nicolas Tabareau. 2019. Eliminating Reflection from Type Theory. In *CPP*, 2019.
- [27] Hongwei Xi. 2003. Applied Type System (extended abstract). In *TYPES*, 2003.

A Formal definition

This appendix defines Ochr completely: its syntax and runtime structures (Section A.1), the machine (Section A.2), observation, Id and conversion (Section A.3), typing (Section A.4) and well-formed environments (Section A.5). Every derivation in the paper can be checked against it. It follows rule set v1.9. The reference checker (Section 8) implements v1.9 except the confinement check of D41, which is in progress; where it differs the text says so. Each group of rules names the checker functions that implement it (in `Machine.lean` unless noted), and says where the checker differs. Section A.6 lists the points where the prose rules left a choice and the reading taken here.

A.1 Syntax and runtime structures

A.1.1 Terms

$t, u, A, B ::=$	$x \mid \text{Prop} \mid \text{Type}_i$	variable, sorts ($i \geq 0$)
	$\mid \Pi(x_1 : A_1 \dots x_n : A_n).B$	dependent function type ($n \geq 1$)
	$\mid \text{fix } f(x_1 : A_1 \dots x_n : A_n) : B \text{ by } x_j := t$	function, recursive on x_j
	$\mid \text{fix } f(x_1 : A_1 \dots x_n : A_n) : B := t$	function, not recursive (λ)
	$\mid t(u_1, \dots, u_n)$	saturated call
	$\mid D \mid C(t_1, \dots, t_k) \mid \text{Unit} \mid () \mid A \times B \mid (t, u) \mid t.1 \mid t.2$	inductive type, constructor; unit, pairs
	$\mid \text{Eq } A \ t \ u \mid \text{refl} \mid J(A, a, b, P, h, t) \mid \top \mid P \wedge Q \mid \langle h, k \rangle$	propositions
	$\mid \&A \mid p \mid \&p \mid p := t$	borrow type; read, borrow, assign
	$\mid \text{let } x = t; u \mid \text{let } x : A = t; u \mid t; u$	sequencing
	$\mid \text{match } p \{C_1(\bar{y}_1) \Rightarrow t_1 \mid \dots \mid C_m(\bar{y}_m) \Rightarrow t_m\}$	case analysis, one arm per constructor
	$\mid \text{ld } A \ t \ u$	equality of computations
	$\mid v$	embedded value
$p, q ::=$	$x \mid *p \mid p.g$	places (g a field name)
$\mathcal{P} ::=$	$(\text{inductive } D := C_1(\overline{g_1 : A_1}) \mid \dots \mid C_m(\overline{g_m : A_m})) \mid (f := \text{fix } \dots) \mid \mathcal{P} \ \mathcal{P}$	programs

Figure 9: Terms and programs. Embedded values occur only in sealed programs and in the code of closures, never in source programs.

Inductive types. A declaration introduces a type D with constructors C_i , each with named fields $g_{i1} : A_{i1}, \dots, g_{ik_i} : A_{ik_i}$. Nat is the declared type $\text{Nat} := \mathbb{Z} \mid S(1 : \text{Nat})$; $S \ t$ abbreviates $S(t)$, and $p.1$ is the predecessor place. The checker builds Nat in, with the same behaviour, and has Bool , lists and trees as declared types (`Check.lean`, `checkInd`).

Binding. In $\Pi(\bar{x} : \bar{A}). B$ and $\text{fix } f(\bar{x} : \bar{A}) : B \dots := t$, each x_i is bound in $A_{i+1} \dots A_n$, in B and in t . With $\text{by } x_j$, f is bound in t and nowhere else; without by , f is not bound at all, and in no case is f bound in $\bar{A}$ or B . $\text{let } x = t; u$ binds x in u . Terms are identified up to renaming of bound variables; the checker uses de Bruijn indices, so this is structural equality (`Syntax.lean`).

Pattern variables are places. The variables $\bar{y}_i = y_1, \dots, y_{k_i}$ of the arm for C_i are not binders: we identify the arm t_i with $t_i[p.g_{i1}/y_1, \dots, p.g_{ik_i}/y_{k_i}]$, which replaces the root y_j of every place in t_i by the field place $p.g_{ij}$ (for Nat , y becomes $p.1$ and $y.1$ becomes $p.1.1$), and write the resolved match $\text{match } p\{C_1 \Rightarrow t_1 \mid \dots \mid C_m \Rightarrow t_m\}$, abbreviated m below. Every definition below (free places, footprints, captures) is on such resolved terms. The checker resolves pattern variables when it parses (`Surface.resolve`).

Borrow types. $\&A$ occurs only as the whole declared type of a variable, a parameter or a function's result, and A contains no $\&$: there are no borrows inside data and no borrows of borrows. The machine also checks this where values are built (`[Pair]`, `[Borrow]`, closure capture).

Type positions. The following positions of a term are *type positions*: the annotation A of $\text{let } x : A = t; u$; the parameter types and the codomain of Π and of fix ; every argument of $\times$, $\&$, $\wedge$ and Eq ; the first argument of Id (whose two sides run on private copies anyway, Section A.3); and the arguments A , a , b and P of J . What stands in a type position is erased (Section A.4.1).

A.1.2 Values, neutrals and types

$v, w, \kappa ::= C(v_1, \dots, v_k) \mid () \mid (v, w)$	data
$\mid \star$	the value of every proof
$\mid \text{borrow}_\ell v \mid \text{loan}_\ell \mid \perp$	borrow, loan, moved-out place
$\mid n \mid F \mid T$	neutrals, function values, types
$n ::= \sigma \mid [t]$	abstract value, sealed program
$F ::= f \mid \langle \bar{\kappa} \vdash \text{fix } f(\bar{x} : \bar{A}) : B \dots := t \rangle$	top-level function, closure
$T ::= s \mid D \mid \text{Unit} \mid T \times T \mid \&T \mid \text{Eq } T \ v \ w \mid \top \mid T \wedge T$	types
$\mid \langle \bar{\kappa} \vdash \Pi(\bar{x} : \bar{A}).B \rangle \mid n$	Π -closure, neutral type
$s ::= \text{Prop} \mid \text{Type}_i$	sorts
$\Omega ::= \varphi_1; \dots; \varphi_k$	frames, oldest first
$\varphi ::= (x : A \mapsto v)^* v^*$	bindings, then temporaries

Figure 10: Values and environments. ℓ ranges over loan labels and σ over abstract values.

A *closure* $\langle \bar{\kappa} \vdash t \rangle$ is code t whose free variables are bound, in order, to the captured values $\bar{\kappa}$; a *Π -closure* is the same for a Π -type. They are the paper’s “closures” and “ Π -types are closures over values”. A top-level function f is a value by itself; its code is in the signature Σ . A neutral is a type exactly when its type is a sort; for instance the sealed program $[\text{Le}(\sigma, \sigma')]$ is a proposition.

A *sealed program* $[t]$ contains a closed term t : it has no free variables, though it may embed values, including abstract values and loans. $[\text{Close}]$ produces sealed programs of the form $[L; C; K]$ with a distinguished *head call* C , which we mark $f(\bar{a})^h$ (the checker’s `Term.call _ _ true`).

Loans. $\text{loans}(v)$ is the set of labels ℓ such that loan_ℓ occurs in v , looking inside sealed programs, closures and types. In an environment Ω , ℓ is *live* if borrow_ℓ occurs in Ω , and *inert* otherwise; inert loans occur only inside the runs of $[\text{Seal}]$. A value is *loan-free* if it contains no live loan, and *borrow-free* if moreover it contains no borrow_ℓ .

A.1.3 Environments and states

An environment $\Omega = \varphi_1; \dots; \varphi_k$ is a stack of frames, oldest first. A frame holds a sequence of *bindings* $x : A \mapsto v$, where the stored type A is omitted for the captured values and parameters bound by the untyped machine, followed by a sequence of *temporaries* v , the values in flight during evaluation (evaluated arguments, the right-hand side of an assignment, a result while its frame is popped). The *positions* π of Ω are its bindings and temporaries, ordered by frame, then bindings before temporaries, then by index; “in the order of Ω ” refers to this order. Variables are resolved in the top frame. We write $\Omega(\pi)$ and $\Omega(x)$ for contents, $\Omega + (x : A \mapsto v)$ for a new binding in the top frame, $\Omega \cdot v$ for a new temporary, and $\Omega; \varphi$ for a pushed frame.

A *state* is an environment together with: Δ , the type of every abstract value; ρ , the refinements made so far, of abstract values and of generalised sealed programs; the goal G while a definition is being checked; and the $[\text{Rec}]$ stack. We write Ω for the whole state and mention the other components only where a rule changes them. Δ and the generalisation records of ρ are *global*: a private copy discards its changes to the environment, the goal and the refinements of abstract values, but keeps every abstract value it introduced and every generalisation it recorded, and fresh names (abstract values and loan labels) are never reused. The global signature Σ records every inductive declaration, and for each top-level definition its Π -type and its code. A program is checked item by item ($[\text{Ind}]$, $[\text{Def}]$, $[\text{Const}]$) against the signature of the items before it.

checker: `Env.lean`, `Check.lean`

A.1.4 Auxiliary definitions

1. **Content and update.** $\text{content}_\Omega(x) = \Omega(x)$; $\text{content}_\Omega(*p) = w$ if $\text{content}_\Omega(p) = \text{borrow}_\ell w$; $\text{content}_\Omega(p.g) = v_j$ if $\text{content}_\Omega(p) = C(v_1, \dots, v_k)$ and g is the j -th field of C ; otherwise $\text{content}_\Omega(p)$ is undefined (so reading

$\mathbf{p.g}$ needs a known constructor with field g , as reading $\mathbf{p.l}$ needs a known $\mathbf{S}$, and $\mathbf{*p}$ needs a borrow). $\Omega[p \mapsto v]$ replaces that sub-value. The *prefixes* of a place are $\text{pre}(x) = \{x\}$, $\text{pre}(*p) = \text{pre}(p) \cup \{*p\}$, $\text{pre}(p.g) = \text{pre}(p) \cup \{p.g\}$; $q \sqsubseteq p$ means $q \in \text{pre}(p)$.

checker: `content, setPlace`

2. **Loans an access must end** ([Access]). For reading, borrowing and assigning,

$$L_{\Omega}^R(p) = \{\ell \mid \text{content}_{\Omega}(q) = \text{loan}_{\ell}, q \in \text{pre}(p)\} \cup \text{loans}(\text{content}_{\Omega}(p)),$$

and for matching, $L_{\Omega}^M(p) = \{\ell \mid \text{content}_{\Omega}(q) = \text{loan}_{\ell}, q \in \text{pre}(p)\} \cup \text{hl}(\text{content}_{\Omega}(p))$, where $\text{hl}([t]) = \text{loans}(t)$ (a loan's position inside a neutral is unknown, so it is treated as the head, D29) and $\text{hl}(v) = \emptyset$ for every other v . Both keep only live labels, ordered from the root of $\mathbf{p}$ outward and then left to right inside the content.

checker: `accessPath, accessInside`

3. **Owners**. For a live label ℓ ,

$$\text{owners}_{\Omega}(\ell) = \bigcup_{\pi: \ell \in \text{loans}(\Omega(\pi))} \begin{cases} \text{owners}_{\Omega}(m) & \text{if } \Omega(\pi) = \text{borrow}_m w \\ \{\pi\} & \text{otherwise,} \end{cases}$$

a set of positions of Ω (well-defined by acyclicity, Section A.5). The owners of a variable are $\text{own}_{\Omega}(x) = \text{owners}_{\Omega}(\ell)$ if $\Omega(x) = \text{borrow}_{\ell} w$, $\emptyset$ if $\Omega(x) = \perp$, and $\{x\}$ otherwise.

checker: `owners (Obs.lean)`

4. **Place occurrences**. $\text{occ}(t)$ is the set of free place occurrences of a resolved term $\mathbf{t}$, each with its kind: read (rd), borrowed (bw), assigned (as) or matched (sc). $\text{occ}(p) = \{(p, \text{rd})\}$, $\text{occ}(\&p) = \{(p, \text{bw})\}$, $\text{occ}(p := t) = \{(p, \text{as})\} \cup \text{occ}(t)$, $\text{occ}(\text{match } p\{C_1 \Rightarrow t_1 \mid \dots \mid C_m \Rightarrow t_m\}) = \{(p, \text{sc})\} \cup \bigcup_i \text{occ}(t_i)$; a binder removes the places rooted at the variable it binds; every other form takes the union over its subterms, including the types and the code of Π and **fix**. A variable x is a *borrow variable* in Ω if its stored type is $\&\mathbf{T}$ or $\Omega(x)$ is a borrow.

checker: `Term.placeOccs, Term.freeOccs (Basic.lean)`

5. **Footprint**.

$$W_{\Omega}(t, u) = \bigcup \{\text{own}_{\Omega}(\text{root}(p)) \mid (p, k) \in \text{occ}(t) \cup \text{occ}(u), k \in \{\text{bw}, \text{as}\} \text{ or } \text{root}(p) \text{ is a borrow variable}\},$$

ordered as Ω . A place rooted at an owned variable that is only read contributes nothing.

checker: `footprint (Obs.lean)`

6. **Substitution and normal form**. For an atom $a \in \{\sigma, \text{loan}_{\ell}\}$, or a neutral $\mathbf{n}$ (for generalisation), $v[w/a]$ replaces every occurrence of $\mathbf{a}$ in $\mathbf{v}$, including inside sealed programs, closures and types, and then restores normal form: every sealed program that changed is replaced by its normal form (`[Seal]`), and every **Eq** and $\mathbf{\Lambda}$ is rebuilt by the smart constructors `eq` and `and` of Section A.3. $\Omega[w/a]$ does this to every value of Ω ; $\Omega[w/a]^+$ also to the stored types, to Δ and to the goal.

checker: `substV, substT, substEnv`

7. **Refinement and generalisation** ([Split], D34). A refinement of an abstract value σ of inductive type $\mathbf{D}$ is $\sigma := C(\sigma_1, \dots, \sigma_k)$ for a constructor $C(g_1 : A_1, \dots, g_k : A_k)$ of $\mathbf{D}$, with σ_j fresh and $\Delta(\sigma_j) = A_j$; $\Omega[\sigma := r]$ is $\Omega[r/\sigma]^+$, and ρ records it. *Generalising* a neutral $n = [t]$ (or an inert loan) at a place of type T takes σ fresh with $\Delta(\sigma) = T$, forms $\Omega[\sigma/n]^+$, and records $n := \sigma$ in ρ , so that it persists: whenever normalisation derives n again it yields $\rho^*(\sigma)$ (`[Seal-stuck]`), and whenever ρ grows every sealed program of the state is re-normalised. The *refined value* $\rho^*(v)$ applies ρ repeatedly, through constructors; the *strict subterms* are $\text{sub}(C(v_1, \dots, v_k)) = \bigcup_j (\{v_j\} \cup \text{sub}(v_j))$ and $\text{sub}(v) = \emptyset$ otherwise.

checker: `refine, expandRefs, strictSubterms, generalizeNeutral, canonNeutral, renormAll`

8. **Drop**. $\text{drop}(\Omega, v) = \Omega'$ where $\Omega \cdot v \rightsquigarrow_{\ell} \Omega' \cdot \perp$ ([End]) if $v = \text{borrow}_{\ell} w$; $\text{drop}(\Omega, v) = \Omega$ if $\mathbf{v}$ is loan-free; otherwise $\text{drop}(\Omega, v) = \text{err}$ (something still borrows a dying value). Popping a frame drops its bindings, newest first, with the call's result held as a temporary of the caller's frame: `pop(\Omega; \varphi, v)`.

checker: `dropTopBind, dropValue, popFrame`

9. **Types of places and values.** $\text{type}_\Omega(x)$ is the stored type of x , or $\text{typeof}(\Omega(x))$ if it has none; $\text{type}_\Omega(*p) = T$ if $\text{type}_\Omega(p) = \&T$; $\text{type}_\Omega(p.g) = A_j$ if $\text{type}_\Omega(p) = D$ and $\text{content}_\Omega(p) = C(\dots)$ with $g_j : A_j$ the field g of C (so a field's type is known once its constructor is). For values: $C(\bar{v})$ has type D for C a constructor of D ; $()$ has **Unit**; (v, w) has $\text{typeof}(v) \times \text{typeof}(w)$; σ has $\Delta(\sigma)$; f has its type in Σ ; a closure $\langle \bar{\kappa} \vdash \text{fix } f(\bar{x} : \bar{A}) : B \dots \rangle$ has the Π -closure $\langle \bar{\kappa} \vdash \Pi(\bar{x} : \bar{A}).B \rangle$; $\text{borrow}_\ell w$ has $\&\text{typeof}(w)$; a type has its sort (Section A.4).

checker: `placeType, valType`

10. **The generic call.** For a Π -closure $\Pi = \langle \bar{\kappa} \vdash \Pi(x_1 : A_1 \dots x_n : A_n).B \rangle$, the *generic arguments* $\bar{a}$ are built left to right. In the frame $\bar{\kappa}, x_1 : T_1 \mapsto a_1, \dots, x_{i-1} : T_{i-1} \mapsto a_{i-1}$, evaluate A_i in a type position to T_i (Section A.4). If $T_i = \&T$, add an owned place $c_i : T \mapsto \text{loan}_{\ell_i}$ to a bottom frame and let $a_i = \text{borrow}_{\ell_i} \sigma_i$ with $\sigma_i : T$ and ℓ_i fresh (this is the argument $\&c_i$ after [Borrow]); if T_i has sort **Prop**, let $a_i = \star$; otherwise $a_i = \sigma_i$ with $\sigma_i : T_i$ fresh. The *generic state* is the frame of owned places $\Gamma(\Pi) = (c_i : T \mapsto \text{loan}_{\ell_i})_{i: T_i = \&T}$, and the *parameter frame* is $\varphi(\Pi) = (\bar{\kappa}, \bar{x} : \bar{T} \mapsto \bar{a})$, each type stored with its binding. The generic call is $F(\bar{a})$ at $\Gamma(\Pi)$.

checker: `checkFix`

A.2 The machine

The machine is the big-step judgement $\langle \Omega, t \rangle \Downarrow r$, where the outcome r is $\langle \Omega', v \rangle$, **stuck** (the run needs to inspect a neutral) or **err** (a borrow error). It is deterministic. **Propagation:** when a premise that evaluates a subterm has outcome **stuck** or **err**, so does the conclusion; the only rules that inspect a **stuck** premise are [App-close], [App-head] and [Seal]. We write $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$ only for successful runs. The machine is the checker's **eval false**; stuck and error outcomes are its exceptions **Fail.stuck** and **Fail.error**.

A.2.1 Borrows

Ending a borrow replaces it by $\perp$ and substitutes its content for its loan everywhere, in normal form (Section A.1.4, item 6). A borrow occurs only as the whole value of a binding or temporary (Section A.5), so it has a position π .

$$\frac{\Omega(\pi) = \text{borrow}_\ell w}{\Omega \xrightarrow[\ell]{\sim} (\Omega[\pi \mapsto \perp])[w/\text{loan}_\ell]} \text{END } \ell$$

[**Access**]. $\text{acc}_p^R(\Omega)$ and $\text{acc}_p^M(\Omega)$ end, one at a time and in the order of item 2 of Section A.1.4, the borrow of the first label of $L_\Omega^R(p)$, respectively $L_\Omega^M(p)$, until that set is empty:

$$\text{acc}_p^X(\Omega) = \begin{cases} \Omega & \text{if } L_\Omega^X(p) = \emptyset \\ \text{acc}_p^X(\Omega') & \text{if } \ell \text{ is the first label of } L_\Omega^X(p) \text{ and } \Omega \xrightarrow[\ell]{\sim} \Omega' \end{cases} \quad (X \in \{R, M\}).$$

Every rule that reads, borrows or assigns a place p first computes acc_p^R , and a match on p computes acc_p^M . This is the borrow checker: an ended borrower holds $\perp$, and every later use of it is an error.

checker: `endBorrow, accessPath, accessInside, accessNeutralHead`

$$\frac{\text{acc}_p^R(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = v \notin \{\perp, \text{borrow}_\ell w\}}{\langle \Omega, p \rangle \Downarrow \langle \Omega_1, v \rangle} \text{READ}$$

$$\frac{\text{acc}_p^R(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = \text{borrow}_\ell w}{\langle \Omega, p \rangle \Downarrow \langle \Omega_1[p \mapsto \perp], \text{borrow}_\ell w \rangle} \text{MOVE} \quad \frac{\text{acc}_p^R(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) \text{ undefined or } \perp}{\langle \Omega, p \rangle \Downarrow \text{err}} \text{READ-ERR}$$

$$\frac{\text{acc}_p^R(\Omega) = \Omega_1 \quad \ell \text{ fresh} \quad \text{content}_{\Omega_1}(p) = v \notin \{\perp, \text{borrow}_m w\}}{\langle \Omega, \&p \rangle \Downarrow \langle \Omega_1[p \mapsto \text{loan}_\ell], \text{borrow}_\ell v \rangle} \text{BORROW}$$

$$\begin{array}{c}
\frac{\text{acc}_p^R(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) \text{ undefined, } \perp \text{ or a borrow}}{\langle \Omega, \&p \rangle \Downarrow \text{err}} \quad \text{BORROW-ERR} \\
\\
\frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \text{acc}_p^R(\Omega_1 \cdot v) = \Omega_2 \cdot v' \quad \text{content}_{\Omega_2}(p) = w \quad \text{drop}(\Omega_2, w) = \Omega_3}{\langle \Omega, p := t \rangle \Downarrow \langle \Omega_3[p \mapsto v'], () \rangle} \quad \text{ASSIGN}
\end{array}$$

In [Assign] the new value travels as a temporary while p is accessed, so that ending a borrow can substitute into it; $\text{drop}(\Omega_2, w)$ ends the old content if it is a borrow, and fails if it still holds a live loan.

checker: `readPlace, borrowPlace, assignPlace`

A.2.2 Sequencing and data

$$\begin{array}{c}
\frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \langle \Omega_1 + (x \mapsto v), u \rangle \Downarrow \langle \Omega_2 + (x \mapsto v'), w \rangle \quad \text{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'}{\langle \Omega, \text{let } x = t; u \rangle \Downarrow \langle \Omega_3, w' \rangle} \quad \text{LET} \\
\\
\frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \text{drop}(\Omega_1, v) = \Omega_2 \quad \langle \Omega_2, u \rangle \Downarrow r}{\langle \Omega, t; u \rangle \Downarrow r} \quad \text{SEQ}
\end{array}$$

The annotated `let x : A = t; u` runs as `let x = t; u`: its annotation is a type position. A discarded value (`t; u`) is dropped like a binding: a borrow ends, and a live loan is an error.

checker: `eval (.letIn, .seq), dropTopBind, dropValue`

$$\begin{array}{c}
\frac{\langle \Omega, \bar{t} \rangle \Downarrow^* \langle \Omega', \bar{v} \rangle}{\langle \Omega, C(\bar{t}) \rangle \Downarrow \langle \Omega', C(\bar{v}) \rangle} \quad \text{CTOR} \quad \frac{}{\langle \Omega, () \rangle \Downarrow \langle \Omega, () \rangle} \quad \text{UNIT} \\
\\
\frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \langle \Omega_1 \cdot v, u \rangle \Downarrow \langle \Omega_2 \cdot v', w \rangle \quad v', w \text{ borrow-free}}{\langle \Omega, (t, u) \rangle \Downarrow \langle \Omega_2, (v', w) \rangle} \quad \text{PAIR} \\
\\
\frac{\langle \Omega, t \rangle \Downarrow \langle \Omega', (v_1, v_2) \rangle}{\langle \Omega, t.i \rangle \Downarrow \langle \Omega', v_i \rangle} \quad \text{PROJ} \quad \frac{\langle \Omega, t \rangle \Downarrow \langle \Omega', n \rangle}{\langle \Omega, t.i \rangle \Downarrow \text{stuck}} \quad \text{PROJ-STUCK}
\end{array}$$

A constructor evaluates its fields left to right, each into a temporary ([Args], below); its field types are borrow-free, so a constructor value holds no borrow.

checker: `evalCore (.ctor, .zero, .succ, .tt, .pair, .fst, .snd)`

A.2.3 Functions, types and proofs

A closure or Π -type captures, when it is formed, the current contents of its free variables $y_1, \dots, y_m$ (in the order of Ω), each accessed as a read: $\Omega_0 = \Omega$, $\Omega_i = \text{acc}_{y_i}^R(\Omega_{i-1})$, $\kappa_i = \text{content}_{\Omega_i}(y_i)$. A captured value may be neither $\perp$ nor a borrow (closures capture no borrows); otherwise the rule fails. The other type formers compute type values; `Eq` and `Λ` use the smart constructors of Section A.3.

$$\begin{array}{c}
\frac{f \in \Sigma}{\langle \Omega, f \rangle \Downarrow \langle \Omega, f \rangle} \quad \text{GLOBAL} \quad \frac{\bar{\kappa} \text{ captured, } \Omega_m}{\langle \Omega, \text{fix } f(\bar{x} : \bar{A}) : B \dots := t \rangle \Downarrow \langle \Omega_m, \langle \bar{\kappa} \vdash \text{fix } f(\bar{x} : \bar{A}) : B \dots := t \rangle \rangle} \quad \text{FIX} \\
\\
\frac{\bar{\kappa} \text{ captured, } \Omega_m}{\langle \Omega, \Pi(\bar{x} : \bar{A}).B \rangle \Downarrow \langle \Omega_m, \langle \bar{\kappa} \vdash \Pi(\bar{x} : \bar{A}).B \rangle \rangle} \quad \text{PI} \quad \frac{}{\langle \Omega, s \rangle \Downarrow \langle \Omega, s \rangle} \quad \text{SORT} \quad \frac{T \in \{\text{D}, \text{Unit}, \top\}}{\langle \Omega, T \rangle \Downarrow \langle \Omega, T \rangle} \quad \text{BASE} \\
\\
\frac{\langle \Omega, A \rangle \Downarrow \langle \Omega_1, T \rangle \quad \langle \Omega_1, B \rangle \Downarrow \langle \Omega_2, T' \rangle}{\langle \Omega, A \times B \rangle \Downarrow \langle \Omega_2, T \times T' \rangle} \quad \text{PROD} \quad \frac{\langle \Omega, A \rangle \Downarrow \langle \Omega', T \rangle}{\langle \Omega, \&A \rangle \Downarrow \langle \Omega', \&T \rangle} \quad \text{REF}
\end{array}$$

$$\begin{array}{c}
\frac{\langle \Omega, A \rangle \Downarrow \langle \Omega_1, T \rangle \quad \langle \Omega_1, a \rangle \Downarrow \langle \Omega_2, v \rangle \quad \langle \Omega_2, b \rangle \Downarrow \langle \Omega_3, w \rangle}{\langle \Omega, \text{Eq } A a b \rangle \Downarrow \langle \Omega_3, \text{eq}(T, v, w) \rangle} \text{Eq} \quad \frac{\langle \Omega, P \rangle \Downarrow \langle \Omega_1, T \rangle \quad \langle \Omega_1, Q \rangle \Downarrow \langle \Omega_2, T' \rangle}{\langle \Omega, P \wedge Q \rangle \Downarrow \langle \Omega_2, \text{and}(T, T') \rangle} \text{AND} \\
\\
\frac{}{\langle \Omega, \text{refl} \rangle \Downarrow \langle \Omega, \star \rangle} \text{REFL} \quad \frac{}{\langle \Omega, \langle h, k \rangle \rangle \Downarrow \langle \Omega, \star \rangle} \text{ANDI} \quad \frac{\langle \Omega, t \rangle \Downarrow r}{\langle \Omega, \text{J}(A, a, b, P, h, t) \rangle \Downarrow r} \text{J}
\end{array}$$

Id **A** **t** **u** is a type former too; its value is given in Section A.3. **J** returns the value of **t** unchanged: its other arguments are in type positions or are the proof **h**, so the machine never runs them (the typing judgement checks them, on private copies). A top-level constant **c** : **A** := **t** evaluates to the value stored in Σ , which is $\star$ when **A** has declared sort **Prop**.

checker: capture, evalCore (.pi, .fix, .sort, .prod, .ref, .eq, .and, .refl, .andI, .prim "J", .const)

Erased terms. Whether an occurrence is erased is read from syntax and declarations, never from a normal form or a computed type (Section A.4.1); the machine needs no types for it, only one flag per binding, saying whether the binding was declared a proof. An erased term runs on a private copy of the environment: its effects are discarded and only its value is kept, which for a proof is $\star$, so a proof need not be run at all.

$$\begin{array}{c}
\frac{t \text{ erased, a declared proof}}{\langle \Omega, t \rangle \Downarrow \langle \Omega, \star \rangle} \text{ERASE-PROOF} \\
\\
\frac{t \text{ erased, a type} \quad \langle \Omega, t \rangle \Downarrow_0 \langle \Omega', T \rangle \text{ confined}}{\langle \Omega, t \rangle \Downarrow \langle \Omega, T \rangle} \text{ERASE-TYPE} \quad \frac{t \text{ erased} \quad \langle \Omega, t \rangle \Downarrow_0 \langle \Omega', v \rangle \text{ not confined}}{\langle \Omega, t \rangle \Downarrow \text{err}} \text{ERASE-ERR}
\end{array}$$

Here $\Downarrow_0$ is the judgement in which the rule for **t**'s own form is applied at the root instead of [Erase-type]. A run of **t** from Ω is *confined* if none of its steps assigns, borrows or moves out of a place rooted at a position of Ω , except a borrow or move that evaluates an argument of an erased call; steps inside an erased subterm are judged by that subterm's own confinement. A proof that the machine skips ([Erase-proof]) was checked to be confined by [T-Erase] when its enclosing definition was checked.

checker: eval, callFn, fnClass, paramFlags, onCopy (Env.lean)

A.2.4 Calls

Arguments are evaluated left to right, each into a temporary of the caller's top frame, so that [Access] can see, and end, the earlier ones:

$$\begin{array}{c}
\frac{}{\langle \Omega, \varepsilon \rangle \Downarrow^* \langle \Omega, \varepsilon \rangle} \text{ARGS-NIL} \quad \frac{\langle \Omega, u \rangle \Downarrow \langle \Omega_1, w \rangle \quad \langle \Omega_1 \cdot w, \bar{u} \rangle \Downarrow^* \langle \Omega_2 \cdot w', \bar{w} \rangle}{\langle \Omega, u \bar{u} \rangle \Downarrow^* \langle \Omega_2, w' \bar{w} \rangle} \text{ARGS} \\
\\
\frac{\langle \Omega, t_0 \rangle \Downarrow \langle \Omega_0, F \rangle \quad \langle \Omega_0 \cdot F, \bar{u} \rangle \Downarrow^* \langle \Omega_1 \cdot F', \bar{w} \rangle \quad \text{no } w_i \text{ is } \perp \quad \langle \Omega_1, F'(\bar{w}) \rangle \Downarrow_{\text{app}} r}{\langle \Omega, t_0(\bar{u}) \rangle \Downarrow r} \text{CALL} \quad \frac{\langle \Omega_0 \cdot F, \bar{u} \rangle \Downarrow^* \langle \Omega_1 \cdot F', \bar{w} \rangle \quad \text{some } w_i = \perp}{\langle \Omega, t_0(\bar{u}) \rangle \Downarrow \text{err}} \text{CALL-ERR}
\end{array}$$

An argument is $\perp$ when a later argument ended its borrow, as in **f**(**&x**, **&x**): it is not of its parameter's type. The judgement $\langle \Omega, F(\bar{w}) \rangle \Downarrow_{\text{app}} r$ applies a function value at its *call point*, the state after the arguments have been evaluated. Let **F** have code **fix** $f(\bar{x} : \bar{A}) : B$ [**by** $x_j := b$ and captured values $\bar{\kappa}$ (none for a top-level **f**)], and let φ be the frame $\bar{\kappa}, [f \mapsto F], \bar{x} \mapsto \bar{w}$, where **f** is bound only with **by**.

$$\begin{array}{c}
\frac{\langle \Omega; \varphi, b \rangle \Downarrow \langle \Omega'; \varphi', v \rangle \quad \text{pop}(\Omega'; \varphi', v) = \langle \Omega'', v' \rangle}{\langle \Omega, F(\bar{w}) \rangle \Downarrow_{\text{app}} \langle \Omega'', v' \rangle} \text{APP} \\
\\
\frac{\langle \Omega; \varphi, b \rangle \Downarrow \text{stuck} \quad \text{the call is not a head call}}{\langle \Omega, F(\bar{w}) \rangle \Downarrow_{\text{app}} \text{close}(\Omega, F, \bar{w})} \text{APP-CLOSE}
\end{array}$$

$$\frac{\langle \Omega; \varphi, b \rangle \Downarrow \text{stuck} \quad \text{the call is a head call } F(\overline{w})^h}{\langle \Omega, F(\overline{w})^h \rangle \Downarrow_{\text{app}} \text{stuck}} \quad \text{APP-HEAD}$$

$$\frac{F = n \text{ a neutral} \quad \text{the call is not a head call}}{\langle \Omega, n(\overline{w}) \rangle \Downarrow_{\text{app}} \text{close}(\Omega, n, \overline{w})} \quad \text{APP-NEUTRAL} \quad \frac{F = n \text{ a neutral}}{\langle \Omega, n(\overline{w})^h \rangle \Downarrow_{\text{app}} \text{stuck}} \quad \text{APP-NEUTRAL-HEAD}$$

[App-close] discards the partial run of the body: close starts again from the call point Ω . The head guard of [Seal] covers neutral-headed calls too: a sealed program whose head call has a neutral head stays as it is.

checker: evalCall, callFn, runBody, popFrame

A.2.5 Closing off

$\text{close}(\Omega, F, \overline{w})$ closes off the call $F(\overline{w})$ at its call point Ω . Its precondition is that every argument is loan-free or is $\text{borrow}_{\ell} u$ with u loan-free; [Access] guarantees it (the checker asserts it). Let $I = \{i \mid w_i = \text{borrow}_{\ell_i} u_i\}$, let c_i ($i \in I$) be fixed names, and let

$$L := (\text{let } c_i = u_i;)_{i \in I} \quad C := F(a_1, \dots, a_n)^h \quad \text{with } a_i = \begin{cases} \&c_i & \text{if } i \in I \\ w_i & \text{otherwise.} \end{cases}$$

The row is chosen by the *declared* codomain $\mathbf{B}$ of F 's type: the codomain term of f 's definition, of the closure's code, of $\Delta(\sigma)$ for an abstract function, or the type of a stuck block.

declared codomain $\mathbf{B}$	result r	each $\text{loan}_{\ell_i}, i \in I$, becomes f_i
Unit	$()$	$[\mathbf{L}; \mathbf{C}; c_i]$
&T (k fresh)	$\text{borrow}_k [\mathbf{L}; \text{let } r = \mathbf{C}; *r]$	$[\mathbf{L}; \text{let } r = \mathbf{C}; *r := \text{loan}_k; c_i]$
any other	$[\mathbf{L}; \mathbf{C}]$	$[\mathbf{L}; \mathbf{C}; c_i]$

Figure 11: [Close]: $\text{close}(\Omega, F, \overline{w}) = \langle \Omega[f_i/\text{loan}_{\ell_i}]_{i \in I}, r \rangle$. The borrows borrow_{ℓ_i} are consumed: at the call point they are no longer in Ω .

checker: closeCall, declKind, resultKind, kindOf

A.2.6 Matching

$$\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = C_i(\overline{v}) \quad \langle \Omega_1, t_i \rangle \Downarrow r}{\langle \Omega, \text{match } p\{C_1 \Rightarrow t_1 \mid \dots \mid C_m \Rightarrow t_m\} \rangle \Downarrow r} \quad \text{MATCH}$$

$$\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) \text{ is a neutral or an inert loan}}{\langle \Omega, \text{match } p\{\dots\} \rangle \Downarrow \text{stuck}} \quad \text{MATCH-STUCK}$$

$$\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) \text{ is undefined or } \perp}{\langle \Omega, \text{match } p\{\dots\} \rangle \Downarrow \text{err}} \quad \text{MATCH-ERR}$$

In arm i the pattern variables are the field places $p.g_{ij}$, so $\&y$ reborrows a field in place (for **Nat**, the predecessor $p.1$).

checker: evalMatch, evalMatchInd, accessNeutralHead

A.2.7 Sealed programs

Values are kept in normal form. The normal form of a sealed program is computed by running it from the empty environment ε (one empty frame), where its head call may not close off ([App-head], D9):

$$\frac{\langle \varepsilon, t \rangle \Downarrow \langle \Omega', v \rangle}{\text{nf}(\lceil t \rceil) = v} \quad \text{SEAL} \quad \frac{\langle \varepsilon, t \rangle \Downarrow \text{stuck}}{\text{nf}(\lceil t \rceil) = \begin{cases} \rho^*(\sigma) & \text{if } \rho \text{ records } \lceil t \rceil := \sigma \\ \lceil t \rceil & \text{otherwise} \end{cases}} \quad \text{SEAL-STUCK} \quad \frac{\langle \varepsilon, t \rangle \Downarrow \text{err}}{\text{nf}(\lceil t \rceil) = \text{err}} \quad \text{SEAL-ERR}$$

Every loan in $\mathbf{t}$ whose borrow lives outside the run is inert in it: [Access] does not end it (it is not live), [Read] copies it (it is loan-free), drop accepts it, and a match on it is stuck, as on an abstract value. An error is a type error at the point that triggered the normalisation. Normalisation happens whenever a sealed program is created or changed: by [Close], by [End] (substituting a loan), and by refinement or generalisation (Section A.1.4, items 6 and 7); a generalised sealed program stays generalised wherever it is derived again. The embedded values of a stuck sealed program are already normal, because substitution normalises inner sealed programs first.

checker: `nfSealed, substV, canonNeutral`

A.2.8 Stuck blocks

A stuck match that is not the body of a call is closed off as a call of an anonymous function of its free places, captured as Rust captures closure variables. This happens only in the typing judgement ([Split], Section A.4), after the match's arms have been checked, which gives its type $\mathbf{B}$ and the set $\mathbf{M}$ of variables that some arm leaves $\perp$ (moves out of).

Let $m = \mathbf{match} \ p\{C_1 \Rightarrow t_1 \mid \dots \mid C_m \Rightarrow t_m\}$ be stuck in Ω , and give each occurrence $(q, k) \in \text{occ}(m)$ a *mode*: **mv** (moved) if q is a whole variable x and either x is a borrow variable and $k \in \{\text{rd}, \text{as}\}$, or $x \in M$; otherwise **ref** (borrowed) if $k \in \{\text{bw}, \text{as}\}$; otherwise **cp** (copied). The *captures* $q_1, \dots, q_n$ are the maximal places of $\text{occ}(m)$, those with no strict prefix among its places (D26: maximal prefixes). Each takes the largest mode of the occurrences at or below it ($\text{cp} < \text{ref} < \text{mv}$), and they are ordered by the position of their root, then by length. Capture q_i becomes a parameter z_i with argument a_i :

$$(z_i : U_i, a_i) = \begin{cases} (z_i : \&\text{type}_\Omega(q_i), \&q_i) & \text{if } q_i \text{ is ref,} \\ (z_i : \text{type}_\Omega(q_i), q_i) & \text{otherwise (a move if its content is a borrow, a copy if not).} \end{cases}$$

The body m' is m with each place $q_i\pi$ renamed to $(*z_i)\pi$ if q_i is **ref**, and to $z_i\pi$ otherwise. Then

$$\text{block}(\Omega, m, B, M) := F_m(a_1, \dots, a_n) \quad \text{where } F_m = \langle \vdash \mathbf{fix} _ (z_1 : U_1 \dots z_n : U_n) : B := m' \rangle,$$

a non-recursive closure with no captured values. Evaluating the block by [Call] gets stuck on the head of m inside F_m 's body and closes off by [App-close], with [Close]'s row chosen by $\mathbf{B}$.

checker: `closeOffMatch, splitThenClose`

A.3 Observation, **Id** and conversion

Eq and **! compute**. Every **Eq** and **!** value is built by the smart constructors

$$\begin{aligned} \text{eq}(T_1 \times T_2, (v_1, v_2), (w_1, w_2)) &= \text{and}(\text{eq}(T_1, v_1, w_1), \text{eq}(T_2, v_2, w_2)) \\ \text{eq}(T, v, w) &= \top \quad \text{if } v \equiv w \text{ (and the first clause does not apply)} \\ \text{eq}(T, v, w) &= \mathbf{Eq} \ T \ v \ w \quad \text{otherwise} \\ \text{and}(\top, P) &= P \quad \text{and}(P, \top) = P \quad \text{and}(P, Q) = P \wedge Q \quad \text{otherwise} \end{aligned}$$

which realise the three conversion rules of Figure 5. The first clause applies only when both sides are pairs; a neutral of product type is not split. **refl** has type $\top$, so by conversion it proves every reflexive equation.

checker: `mkEqM, mkAnd (Basic.lean)`

Observation. Let $W = \pi_1 < \dots < \pi_k$ be positions of Ω . The observation of $\mathbf{t}$ at Ω on $\mathbf{W}$ runs $\mathbf{t}$ on a private copy of Ω , ends every remaining borrow, and reads off the result and the footprint. A borrow-typed result (only D30 observes one, since **Id** needs a borrow-free type) is observed as its current content, after a given abstract value σ_w is written through it, so that its owners show where it points:

$$\begin{array}{c}
\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad v \text{ is not a borrow} \\
\Omega_1 \cdot v \rightsquigarrow^* \Omega_2 \cdot v' \text{ with no borrow left in } \Omega_2 \cdot v' \quad \text{OBS} \\
\hline
\llbracket t \rrbracket_\Omega^W = \text{tuple}(v', \Omega_2(\pi_1), \dots, \Omega_2(\pi_k)) \\
\\
\Omega \vdash t \Downarrow \text{borrow}_k u : \&T \dashv \Omega_1 \\
\Omega_1 \cdot u \cdot \text{borrow}_k \sigma_w \rightsquigarrow^* \Omega_2 \cdot u' \cdot \perp \text{ with no borrow left} \quad \text{OBS-BORROW} \\
\hline
\llbracket t \rrbracket_\Omega^{W; \sigma_w} = \text{tuple}(u', \Omega_2(\pi_1), \dots, \Omega_2(\pi_k))
\end{array}$$

where $\rightsquigarrow^*$ repeatedly ends the first borrow in the order of Ω (so in [Obs-borrow] σ_w reaches the owners of loan_k , as $*r := \sigma_w$ would), $\text{tuple}(v) = v$ and $\text{tuple}(v, w_1, \dots, w_k) = (v, (w_1, (\dots, w_k)))$. The run of $\mathbf{t}$ pops every frame it pushes, so the positions of $\mathbf{W}$ are still positions of Ω_2 . A stuck match in $\mathbf{t}$ is closed off by [Split] (Section A.4), so an observation exists unless $\mathbf{t}$ has an error.

checker: observe, endAll, tupleVal (Obs.lean)

Id computes. Both sides run from the same Ω , on independent copies:

$$\begin{array}{c}
\Omega \vdash A \Downarrow T \text{ type, } T \text{ contains no } \& \quad W = W_\Omega(t, u) = \pi_1 < \dots < \pi_k \\
\llbracket t \rrbracket_\Omega^W \text{ with } t : A_t \equiv T \quad \llbracket u \rrbracket_\Omega^W \text{ with } u : A_u \equiv T \quad T_W = \text{type}_\Omega(\pi_1) \times \dots \times \text{type}_\Omega(\pi_k) \quad \text{ID} \\
\hline
\Omega \vdash \text{Id } A \ t \ u \Downarrow \text{eq}(T \times T_W, \llbracket t \rrbracket_\Omega^W, \llbracket u \rrbracket_\Omega^W) : \text{Prop} \dashv \Omega
\end{array}$$

When $\mathbf{W}$ is empty, $T \times T_W$ is just $\mathbf{T}$ and the observations are just results.

checker: idType, footprint, tupleType (Obs.lean)

Conversion. Definitional equality is equality of normal forms, and values are always kept in normal form, so conversion $v \equiv w$ is the least relation closed under the following rules. It is the relation of every “ $\equiv$ ” premise in Section A.4 and of the reflexive clause of eq.

- [Conv-refl] $v \equiv v$, where values are identified up to renaming of bound variables (binder names are ignored).
- [Conv-cong] Two values with the same outermost former ($\mathbf{S}$, pair, borrow, $\times$, $\&$, $\mathbf{Eq}$, $\mathbf{\Lambda}$, a sealed program, a closure, a Π -closure) are convertible if their immediate components are pairwise convertible: terms (the code of closures and Π -closures, the programs inside sealed programs) component by component, with embedded values compared by $\equiv$. So Π -types are compared under binders by their captured values and their code.
- [Conv-fun] Two function values $\mathbf{F}$, $\mathbf{G}$ are convertible if their Π -types are, their captured values are pairwise convertible (none, for a top-level function), and at the generic call of their Π -type (Section A.1.4, item 10), with the same generic state Γ , arguments $\bar{a}$ and owned places $\bar{c}$, their observations agree: $\llbracket F(\bar{a}) \rrbracket_\Gamma^{\bar{c}} \equiv \llbracket G(\bar{a}) \rrbracket_\Gamma^{\bar{c}}$. For a codomain $\&\mathbf{T}$ both observations use [Obs-borrow] with one shared fresh $\sigma_w : T$, so $\lambda(x \ y : \&\mathbf{Nat}). x$ and $\lambda(x \ y : \&\mathbf{Nat}). y$ differ: one leaves (σ_w, σ_y) in the owned places, the other (σ_x, σ_w) . So a function value’s normal form is its captured values together with the observation of its generic call, and two functions with different effects are never convertible.

There is no η rule and no η for $\mathbf{Unit}$ (an abstract $\sigma : \mathbf{Unit}$ is not $()$). Proof irrelevance needs no rule: every proof value is $\star$. The relation is *least*: a function value that occurs in its own generic observation, as the head of a sealed program when its body is stuck at its generic call, is compared there by [Conv-refl] and [Conv-cong]. Read coinductively, [Conv-fun] would identify any two closures whose bodies are stuck at the generic call, such as $\lambda(x : \&\mathbf{Nat}). \text{match } *x \{ Z \Rightarrow () \mid S _ \Rightarrow *x := Z \}$ and $\lambda(x : \&\mathbf{Nat}). \text{match } *x \{ Z \Rightarrow *x := S \ Z \mid S _ \Rightarrow () \}$, and $\mathbf{J}$ along that identification proves $\mathbf{Eq \ Nat \ (S \ Z) \ Z}$.

checker: conv, convT, convFn, mkEqM

The checker implements this relation (`conv`, `convT`, `convFn`), with [Obs-borrow] for borrow-returning functions. It answers “not convertible” when a comparison needs itself again (`convStack`), which is the least-relation reading (RULES P1).

A.4 Typing

The typing judgement $\Omega \vdash t \Downarrow v : A \dashv \Omega'$ says that from Ω the term t runs to the value v , of type A , leaving Ω' ; its only other outcome is **err**, a type error. It is the machine with types: each rule below performs the step of the machine rule of the same name and also computes a type. Where the machine would be stuck on the checked program, typing splits instead ([Split]), so the typing judgement is never stuck. Callee bodies run in the machine, where stuck calls close off ([App-close]). We write $\Omega \vdash A \Downarrow T : s \dashv \Omega'$ with s a sort and Ω' discarded (a type position). The checker's typing judgement is **eval true**.

Sorts. Sorts are ordered $\text{Prop} < \text{Type}_0 < \text{Type}_1 < \dots$, and $s \sqcup s'$ is the larger. Universes are *not cumulative*: a type has exactly one sort, and no rule converts between sorts. The sort of a type value is: Type_s for declared inductive types (**Nat** among them), **Unit** and $\&\mathbb{T}$; $\text{Type}_0 \sqcup \text{sort}(T) \sqcup \text{sort}(T')$ for $T \times T'$; **Prop** for **Eq**, τ and $\wedge$; Type_0 for **Prop** and Type_{i+1} for Type_i ; for a Π -closure, **Prop** if its codomain at the generic call has sort **Prop** (impredicativity), and otherwise the largest sort of its parameter types and codomain there; $\Delta(\sigma)$ for an abstract σ whose type is a sort; and for a sealed program $[L; C]$ whose head call's function has a declared codomain that is literally a sort s , the sort s .

checker: sortOf, sealedSort?, isPropV

A.4.1 Erasure

An occurrence of a term is *erased* when:

1. it stands in a type position (Section A.1), or inside one; or
2. it is a type former (a sort, Π , an inductive type, **Unit**, $\times$, $\&$, **Eq**, τ , $\wedge$, **Id**), since every type is formed on a private copy; or
3. it is a call whose callee *returns types* or *returns proofs*; or
4. it is a *declared proof*, judged from syntax alone: **refl**; $\langle h, k \rangle$; a **J** whose motive is syntactically a function into **Prop**, or whose last argument is a declared proof; a call whose callee returns proofs; a variable whose binding is flagged as a proof (a parameter whose declared type has declared sort **Prop**, or a **let** whose right-hand side is a declared proof; captured values are never flagged); a **fix** whose Π -type returns proofs; **let** $x = t; u, t; u$ or a **match** run directly when its tail (u , or the arm taken) is a declared proof; a stuck block when every arm is; or the bound term of **let** $x : A = t; u$ when A has declared sort **Prop**.

A function value, top-level or local, *returns types* if the codomain term of its Π -type is syntactically a sort, and *returns proofs* if that term's *declared sort* is **Prop**. A type term has declared sort **Prop** when it is **Id**, **Eq**, τ or $\wedge$; a Π into such a term; a variable declared $: \text{Prop}$; a call whose head's codomain term has declared sort **Prop**; a **let**, $;$ or **match** whose tail does; or an ascription at **Prop**. Nothing is normalised: $U(n)$ with $U : \Pi(n : \text{Nat})$. Type_s has declared sort Type_s , whatever $U(n)$ computes to, and $V(Z)$ with $V : \Pi(n : \text{Nat})$. $U(n)$ does not have declared sort **Prop** although it computes to τ . A stuck block is erased exactly when *each* of its arms, as checked by [Split], is a declared proof; it is never erased by clause 3 applied to its codomain, nor by its type inferred from the arms, since both are computed. No clause consults the type the typing judgement computes: mixing the computed and the declared readings admits **BoomG**, and reading a variable's value admits **BoomH**. A block that is not erased is always safe: its sealed programs re-run the arms, which make their own decisions. Every clause reads syntax, a declaration or a sort, so the two paths of Lemma 7 take the same decisions. Notes 1, 2 and 4 of Section A.6 show what goes wrong otherwise.

$$\frac{t \text{ erased} \quad \Omega \vdash_0 t \Downarrow v : A \dashv \Omega' \text{ by a confined run}}{\Omega \vdash t \Downarrow v^\bullet : A \dashv \Omega} \text{ T-ERASE}$$

Here $v^\bullet = \star$ if t is a declared proof (clause 4) and $v^\bullet = v$ otherwise, $\vdash_0$ applies the rule for t 's own form at the root, and a run that is not confined (Section A.2, [Erase-err]) is a type error. So an erased term is typed like any other term, on a private copy of the environment, and leaves no trace. Confinement is redundant for a correctly classified term, since the private copy already discards its effects; it is a fail-safe: if the two evaluation paths ever disagreed about whether a term is erased, the path that erases a term with outside effects would reject it,

instead of silently discarding effects that the other path keeps. A proof may still mutate its own locals, and may hand outer places to other erased calls, as `AddMZero`'s recursive call `AddMZero(&p)` does.

checker: `eval, fnClass, propDecl, typeClass, jErased, evalType, onCopy` (Env.lean)

The checker implements these clauses (`fnClass`, `propDecl`, and a proof flag on every binding, `paramFlags`). `propDecl` is incomplete: a codomain such as `q.1`, projecting a proposition out of a captured pair, is classed as data; the classification is still the same on both paths.

A.4.2 Places, sequencing and data

$$\begin{array}{c}
\frac{\langle \Omega, p \rangle \Downarrow \langle \Omega', v \rangle}{\Omega \vdash p \Downarrow v : \text{type}_\Omega(p) \dashv \Omega'} \text{ T-READ} \quad \frac{\langle \Omega, \&p \rangle \Downarrow \langle \Omega', v \rangle \quad \text{type}_\Omega(p) = T \text{ contains no } \&}{\Omega \vdash \&p \Downarrow v : \&T \dashv \Omega'} \text{ T-BORROW} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad A \equiv \text{type}_{\Omega_1}(p) \quad \text{acc}_p^R(\Omega_1 \cdot v) = \Omega_2 \cdot v' \quad \text{drop}(\Omega_2, \text{content}_{\Omega_2}(p)) = \Omega_3}{\Omega \vdash p := t \Downarrow () : \text{Unit} \dashv \Omega_3[p \mapsto v']} \text{ T-ASSIGN} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \Omega_1 + (x : A \mapsto v) \vdash u \Downarrow w : B \dashv \Omega_2 + (x : A' \mapsto v') \quad \text{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'}{\Omega \vdash \text{let } x = t; u \Downarrow w' : B \dashv \Omega_3} \text{ T-LET} \\
\\
\frac{\Omega \vdash A \Downarrow T \text{ type} \quad \Omega \vdash^T t \Downarrow v : T' \dashv \Omega_1 \quad T' \equiv T \quad \Omega_1 + (x : T \mapsto v) \vdash u \Downarrow w : B \dashv \Omega_2 + (x : T'' \mapsto v') \quad \text{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'}{\Omega \vdash \text{let } x : A = t; u \Downarrow w' : B \dashv \Omega_3} \text{ T-LET-ANN} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \text{drop}(\Omega_1, v) = \Omega_2 \quad \Omega_2 \vdash u \Downarrow w : B \dashv \Omega_3}{\Omega \vdash t; u \Downarrow w : B \dashv \Omega_3} \text{ T-SEQ}
\end{array}$$

A binding stores the type of its value, and [Split] refines stored types, so `A'` in [T-Let] is `A` as refined. In [T-Let-ann], $\vdash^T$ checks `t` against `T`: if `t` is a match, it is [Split] with annotation `T`.

checker: `placeType, evalCore` (`.place`, `.borrow`, `.assign`, `.letIn`, `.seq`, `.ascribe`)

$$\begin{array}{c}
\frac{\text{C}(g_1 : A_1, \dots, g_k : A_k) \text{ a constructor of } D \quad \Omega \vdash \bar{t} \Downarrow^* \bar{v} : \bar{A}' \dashv \Omega' \quad A'_j \equiv A_j}{\Omega \vdash \text{C}(\bar{t}) \Downarrow \text{C}(\bar{v}) : D \dashv \Omega'} \text{ T-CTOR} \quad \frac{}{\Omega \vdash () \Downarrow () : \text{Unit} \dashv \Omega} \text{ T-UNIT} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \Omega_1 \cdot v \vdash u \Downarrow w : B \dashv \Omega_2 \cdot v' \quad A, B \text{ contain no } \&}{\Omega \vdash (t, u) \Downarrow (v', w) : A \times B \dashv \Omega_2} \text{ T-PAIR} \quad \frac{\Omega \vdash t \Downarrow (v_1, v_2) : A_1 \times A_2 \dashv \Omega'}{\Omega \vdash t.i \Downarrow v_i : A_i \dashv \Omega'} \text{ T-PROJ}
\end{array}$$

The typed argument judgement $\Downarrow^*$ is that of [T-Call]. A projection of a neutral pair is a type error in the typing judgement (there are no neutral projections in the core); in the machine it is stuck.

checker: `evalCore` (`.ctor`, `.zero`, `.succ`, `.tt`, `.pair`, `.fst`, `.snd`)

A.4.3 Type formers and proofs

All type formers are erased, so each of these rules runs on a private copy ([T-Erase]).

$$\begin{array}{c}
\frac{}{\Omega \vdash \text{Prop} \Downarrow \text{Prop} : \text{Type}_0 \dashv \Omega} \text{ T-SORT} \quad \frac{}{\Omega \vdash \text{Type}_i \Downarrow \text{Type}_i : \text{Type}_{i+1} \dashv \Omega} \text{ T-TYPE} \\
\\
\frac{(T, s) \in \{(D, \text{Type}_0), (\text{Unit}, \text{Type}_0), (\top, \text{Prop})\}}{\Omega \vdash T \Downarrow T : s \dashv \Omega} \text{ T-BASE} \quad \frac{\Omega \vdash A \Downarrow T : s \quad \Omega \vdash B \Downarrow T' : s'}{\Omega \vdash A \times B \Downarrow T \times T' : \text{Type}_0 \sqcup s \sqcup s'} \text{ T-PROD}
\end{array}$$

$$\begin{array}{c}
\frac{\Omega \vdash A \Downarrow T \text{ type} \quad T \text{ contains no } \&}{\Omega \vdash \&A \Downarrow \&T : \mathbf{Type}_0} \text{ T-REF} \quad \frac{\Omega \vdash P \Downarrow T : \mathbf{Prop} \quad \Omega \vdash Q \Downarrow T' : \mathbf{Prop}}{\Omega \vdash P \wedge Q \Downarrow \text{and}(T, T') : \mathbf{Prop}} \text{ T-AND} \\
\\
\frac{\Omega \vdash A \Downarrow T \text{ type} \quad \Omega \vdash a \Downarrow v : T_a \equiv T \quad \Omega \vdash b \Downarrow w : T_b \equiv T}{\Omega \vdash \mathbf{Eq} A a b \Downarrow \text{eq}(T, v, w) : \mathbf{Prop}} \text{ T-EQ} \\
\\
\frac{\langle \Omega, \Pi(\bar{x} : \bar{A}).B \rangle \Downarrow \langle \Omega', \Pi \rangle \quad \text{sort}(\Pi) = s}{\Omega \vdash \Pi(\bar{x} : \bar{A}).B \Downarrow \Pi : s \dashv \Omega'} \text{ T-PI}
\end{array}$$

In [T-Eq] the sides are typed one after the other on the same private copy (the whole **Eq** is a type); in [T-Pi] the sort is computed at the generic arguments. The rule for **Id** is [Id] of Section A.3.

checker: `evalCore (.sort, .nat, .unit, .top, .prod, .ref, .and, .eq, .pi, .id), sortOf`

$$\begin{array}{c}
\frac{}{\Omega \vdash \mathbf{refl} \Downarrow \star : \top \dashv \Omega} \text{ T-REFL} \quad \frac{\Omega \vdash h \Downarrow \star : P \dashv \Omega_1 \quad \Omega_1 \vdash k \Downarrow \star : Q \dashv \Omega_2}{\Omega \vdash \langle h, k \rangle \Downarrow \star : \text{and}(P, Q) \dashv \Omega_2} \text{ T-ANDI} \\
\\
\frac{\Omega \vdash A \Downarrow T \text{ type} \quad \Omega \vdash a \Downarrow v_a : T_a \equiv T \quad \Omega \vdash b \Downarrow v_b : T_b \equiv T \quad \Omega \vdash P \Downarrow F : \Pi \quad \Omega \vdash h \Downarrow \star : H \equiv \text{eq}(T, v_a, v_b) \quad \Omega \vdash F(v_a) \Downarrow P_a : s \quad \Omega \vdash F(v_b) \Downarrow P_b : s \quad \Omega \vdash t \Downarrow v : T_t \dashv \Omega' \quad T_t \equiv P_a}{\Omega \vdash \mathbf{J}(A, a, b, P, h, t) \Downarrow v : P_b \dashv \Omega'} \text{ T-J}
\end{array}$$

refl proves $\top$, and so by conversion every reflexive equation. **J** takes its endpoints explicitly, because **Eq A a a** computes to $\top$ and no longer records them. Its motive may have any sort; with a motive into **Prop**, **J** is a proof and is erased. **A**, **a**, **b** and **P** are in type positions and **h** is a proof, so all five are typed on private copies, and the motive's calls $F(v_a)$, $F(v_b)$ are erased calls.

checker: `evalCore (.refl, .andI, .prim "J")`

A.4.4 Functions and calls

$$\begin{array}{c}
\frac{f \in \Sigma}{\Omega \vdash f \Downarrow f : \Sigma(f).\text{type} \dashv \Omega} \text{ T-GLOBAL} \\
\\
\frac{c \in \Sigma}{\Omega \vdash c \Downarrow \Sigma(c).\text{value} : \Sigma(c).\text{type} \dashv \Omega} \text{ T-CONST} \quad \frac{\langle \Omega, \mathbf{fix} \dots \rangle \Downarrow \langle \Omega', F \rangle \quad \vdash F \text{ ok}}{\Omega \vdash \mathbf{fix} \dots \Downarrow F : \text{typeof}(F) \dashv \Omega'} \text{ T-FIX}
\end{array}$$

A **fix** is checked where it is formed, by [Def] at its own generic call, with its captured values; its value is $\star$ if its Π -type returns proofs.

checker: `evalCore (.const, .fix), checkFix`

[**Call-type**]. At the call point Ω of a call with function type $\Pi = \langle \bar{\kappa} \vdash \Pi(x_1 : A_1 \dots x_n : A_n).B \rangle$ and arguments $\bar{w}$ of types $\bar{A}'$, push the frame of captured values and bind the parameters one by one, on a private copy:

$$\frac{\Omega; (\bar{\kappa}, x_1 : T_1 \mapsto w_1, \dots, x_{i-1} : T_{i-1} \mapsto w_{i-1}) \vdash A_i \Downarrow T_i \text{ type} \quad A'_i \equiv T_i \quad (1 \leq i \leq n) \quad \Omega; (\bar{\kappa}, \bar{x} : \bar{T} \mapsto \bar{w}) \vdash B \Downarrow T_B \text{ type}}{\text{callty}_\Omega(\Pi, \bar{w} : \bar{A}') = T_B} \text{ CALL-TYPE}$$

The free variables of **B** other than $\bar{x}$ were captured when Π was formed; the caller's frames below are not in scope, but the footprints and owners of observations in **B** see them, which is how an induction hypothesis sees its caller's context. A borrow argument is moved into its parameter.

checker: `callType`

Typed arguments are evaluated like [Args], each into a temporary, collecting their types: $\Omega \vdash \bar{u} \Downarrow^* \bar{w} : \bar{A}' \dashv \Omega'$.

$$\frac{\Omega \vdash t_0 \Downarrow F : \Pi \dashv \Omega_0 \quad \Omega_0 \cdot F \vdash \bar{u} \Downarrow^* \bar{w} : \bar{A}' \dashv \Omega_1 \cdot F' \quad \text{no } w_i \text{ is } \perp \quad \text{callty}_{\Omega_1}(\Pi, \bar{w} : \bar{A}') = B' \quad \text{rec}_{\Omega_1}(F', \bar{w}) \quad \langle \Omega_1, F'(\bar{w}) \rangle \Downarrow_{\text{app}} \langle \Omega', v \rangle}{\Omega \vdash t_0(\bar{u}) \Downarrow v : B' \dashv \Omega'} \text{ T-CALL}$$

$$\frac{\Omega \vdash t_0 \Downarrow F : \Pi \dashv \Omega_0 \quad \Omega_0 \cdot F \vdash \bar{u} \Downarrow^* \bar{w} : \bar{A}' \dashv \Omega_1 \cdot F' \quad \text{no } w_i \text{ is } \perp \quad \text{callty}_{\Omega_1}(\Pi, \bar{w} : \bar{A}') = B' \quad F' \text{ returns proofs} \quad \text{rec}_{\Omega_1}(F', \bar{w})}{\Omega \vdash t_0(\bar{u}) \Downarrow \star : B' \dashv \Omega_1} \text{ T-CALL-PROOF}$$

[T-Call-proof] applies to a call whose callee returns proofs (Section A.4.1), and [T-Erase] then discards Ω_1 : a proof call's arguments are evaluated and checked, and its body is not run. In [T-Call] the callee's body runs in the machine ($\Downarrow_{\text{app}}$), since the callee was checked by its own [Def]; a call with a neutral head closes off at once.

checker: `evalCall`, `callFn`, `callType`

[Rec]. The [Rec] stack holds an entry (F, j, σ_j) for every enclosing function being checked that declares `by` x_j , where σ_j is the entry value of x_j at its generic call (through the borrow, for $\&\text{D}$). $\text{rec}_{\Omega}(F, \bar{w})$ holds when, for every entry (F, j, σ_j) of the stack for this F , the argument w_j , or its content if $w_j = \text{borrow}_\ell u$, is in $\text{sub}(\rho^*(\sigma_j))$: a strict subterm, through any field, of the entry value as refined so far. Entries stay on the stack while nested functions and block arms are checked, so recursive calls there are checked too.

checker: `recCheck`, `headOnly`

A.4.5 Matches and case splitting

Let $m = \text{match } p\{C_1 \Rightarrow t_1 \mid \dots \mid C_m \Rightarrow t_m\}$ match on a place of inductive type D with constructors $C_1, \dots, C_m$, and let $r_i = C_i(\sigma_{i1}, \dots, \sigma_{ik_i})$ be the refinement to C_i with fresh field values (Section A.1.4, item 7). On a constructor the typing judgement takes the arm, as the machine does, and checks only that arm. On an abstract value it splits ([Split], D15, D22): each arm is checked under its refinement, and the match is then closed off as a stuck block, from which the rest of the program is checked once.

$$\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = C_i(\bar{w}) \quad \Omega_1 \vdash t_i \Downarrow v : A \dashv \Omega'}{\Omega \vdash m \Downarrow v : A \dashv \Omega'} \text{ T-MATCH}$$

$$\frac{\begin{array}{l} \text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = \sigma \quad \Omega_1[\sigma := r_i] \vdash t_i \Downarrow v_i : A_i \dashv \Omega_i \quad (1 \leq i \leq m) \\ B = T \text{ with } A_i \equiv T[r_i/\sigma] \text{ for all } i, \quad \text{or, without annotation, } B = A_1 \equiv \dots \equiv A_m \\ M = \{x \mid \Omega_1(x) \neq \perp, \text{ some } \Omega_i(x) = \perp\} \quad \langle \Omega_1, \text{block}(\Omega_1, m, B, M) \rangle \Downarrow \langle \Omega', v \rangle \end{array}}{\Omega \vdash m \Downarrow v : B \dashv \Omega'} \text{ SPLIT}$$

$$\frac{\begin{array}{l} \text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = n \text{ a sealed program or an inert loan} \\ \sigma \text{ fresh, } \Delta(\sigma) = \text{type}_{\Omega_1}(p) \quad \Omega_1[\sigma/n]^+ \text{ with } n := \sigma \text{ recorded in } \rho \quad \vdash m \Downarrow v : B \dashv \Omega' \end{array}}{\Omega \vdash m \Downarrow v : B \dashv \Omega'} \text{ SPLIT-GEN}$$

In [Split] every arm is checked from Ω_1 , and the arms' states and values are discarded; each refinement is applied to the environment, the goal and every stored type. T is the annotation of an enclosing `let` $x : \text{T} = m$ (checking mode, $\vdash^T$), whose type is then T ; without one, the arms' types must be convertible. The block is run by the machine, from the unrefined Ω_1 . [Split-gen] generalises a neutral head first (Lean's `generalize`): it replaces the neutral everywhere in Ω , the goal, the stored types and the annotation, and records it in ρ , so that the replacement persists when the neutral is derived again; the record is global, and survives a private copy in which it was made. Types are terms, so a match inside a type is checked in the same way.

checker: `evalMatch`, `evalMatchInd`, `splitArmsThenClose`, `ctorRefinement`, `generalizeNeutral`, `refine`

Tail position. The body of a definition is checked by a judgement $\Omega \vdash t \rightsquigarrow \mathcal{L}$ whose result is the finite set $\mathcal{L}$ of the *leaves* of its case tree: triples (Ω', v, A) , each in the state of its own path, with the goal refined along that path. A match is in tail position when only trailing drops follow it.

$$\begin{array}{c}
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = \sigma \quad \Omega_1[\sigma := r_i] \vdash t_i \rightsquigarrow \mathcal{L}_i \quad (1 \leq i \leq m)}{\Omega \vdash m \rightsquigarrow \mathcal{L}_1 \cup \dots \cup \mathcal{L}_m} \text{TAIL-SPLIT} \\
\\
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = n \quad \Omega_1[\sigma/n]^+ \vdash m \rightsquigarrow \mathcal{L}}{\Omega \vdash m \rightsquigarrow \mathcal{L}} \text{TAIL-GEN} \\
\\
\frac{\text{acc}_p^M(\Omega) = \Omega_1 \quad \text{content}_{\Omega_1}(p) = C_i(\bar{w}) \quad \Omega_1 \vdash t_i \rightsquigarrow \mathcal{L}}{\Omega \vdash m \rightsquigarrow \mathcal{L}} \text{TAIL-MATCH} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \Omega_1 + (x : A \mapsto v) \vdash u \rightsquigarrow \mathcal{L}}{\Omega \vdash \text{let } x = t; u \rightsquigarrow \{\text{pop}_x(\ell) \mid \ell \in \mathcal{L}\}} \text{TAIL-LET} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega_1 \quad \text{drop}(\Omega_1, v) = \Omega_2 \quad \Omega_2 \vdash u \rightsquigarrow \mathcal{L}}{\Omega \vdash t; u \rightsquigarrow \mathcal{L}} \text{TAIL-SEQ} \\
\\
\frac{\Omega \vdash t \Downarrow v : A \dashv \Omega' \quad t \text{ is not a match, let or sequence}}{\Omega \vdash t \rightsquigarrow \{(\Omega', v, A)\}} \text{TAIL-END}
\end{array}$$

Here $\text{pop}_x(\Omega_2 + (x : A' \mapsto v'), w, B) = (\Omega_3, w', B)$ with $\text{drop}(\Omega_2 \cdot w, v') = \Omega_3 \cdot w'$, and an annotated **let** checks t against its annotation as in [T-Let-ann]. In [Tail-gen], σ is fresh and $n := \sigma$ is recorded in ρ , as in [Split-gen]. An error on any path is an error.

checker: `checkTail`

A.4.6 Definitions

[Def]. Let F be a top-level function or a closure with captured values $\bar{\kappa}$ and code **fix** $f(x_1 : A_1 \dots x_n : A_n) : B$ **by** $x_j := b$, and let $\Pi = \text{typeof}(F)$, with generic state $\Gamma = \Gamma(\Pi)$, parameter frame $\varphi = \varphi(\Pi)$ and parameter types $\bar{T}$ (Section A.1.4, item 10). Let φ^+ be φ with $f : \Pi \mapsto F$ added when F declares **by**.

$$\frac{\begin{array}{l} f \text{ occurs in } b \text{ only as the head of a call } [\text{by } x_j] : T_j \in \{\mathbf{D}, \&\mathbf{D}\} \\ \Gamma; \varphi \vdash B \Downarrow G \text{ type} \quad \Gamma; \varphi^+ \vdash b \rightsquigarrow \mathcal{L} \text{ with goal } G \text{ and the [Rec] entry } (F, j, \sigma_j) \text{ pushed} \\ \text{for every } (\Omega', v, A) \in \mathcal{L} \text{ with refined goal } G' : \text{pop}(\Omega', v) \text{ succeeds and } A \equiv G' \end{array}}{\vdash F \text{ ok}} \text{DEF}$$

In the first premise, f must also occur only as a call head inside nested functions and block arms. The goal G is the [Call-type] of the generic call $F(\bar{a}) : B$ evaluated with each parameter bound to its generic argument, borrow parameters to borrows of the fresh owned places c_i , proof parameters to $\star$. Parameter types are evaluated left to right and stored with their bindings, where [Split] refines them. Popping the body's frame drops its bindings, so each borrow parameter ends and its owned place receives the final content; the result's type must then convert to the goal as refined on that path. A top-level f is added to Σ , with its type, code and erasure flag, before its body is checked, so that its recursive calls resolve; it is kept only if the check succeeds.

checker: `checkFix, checkDef (Check.lean)`

$$\frac{\begin{array}{l} \mathbf{D} \notin \Sigma \quad \text{for every field } g_{ij} : A_{ij} : A_{ij} \text{ is built from } \mathbf{D}, \text{ declared inductive types, Unit and } \times \\ \Sigma + \mathbf{D} \vdash A_{ij} \Downarrow T_{ij} : \text{Type}_0 \end{array}}{\vdash (\text{inductive } \mathbf{D} := C_1(\bar{g}_1 : A_1) \mid \dots \mid C_m(\bar{g}_m : A_m)) \text{ ok}} \text{IND} \\
\\
\frac{\varepsilon \vdash A \Downarrow T \text{ type} \quad \varepsilon \vdash t \Downarrow v : T' \dashv \Omega' \quad T' \equiv T \quad \Sigma(c) = (T, v^\bullet)}{\vdash (c : A := t) \text{ ok}} \text{CONST}$$

Field types are first-order data: they may mention the type being declared, but contain no Π and no $\&$, which is strict positivity in its simplest form (note 3 of Section A.6). A definition without parameters is a constant: its

body is checked against its type from the empty environment, and its value is stored ($v^\bullet = \star$ if T has declared sort **Prop**). A definition may not mention itself in its type.

checker: `checkInd, checkDef (Check.lean)`

A.5 Well-formed environments

A state (Ω, Δ, Σ) is *well formed* when the following hold. They are the four conditions of Section 7.1, spelled out, plus typing.

1. **Borrows are unique and outermost.** Each label ℓ has at most one borrow_ℓ in Ω , and a borrow occurs only as the whole value of a binding or temporary: never inside data, a sealed program, a closure or a type.
2. **Loans are bound.** Every loan_ℓ in Ω has its borrow_ℓ in Ω , except inside a `[Seal]` run, where a loan whose borrow lies outside the run is inert. Outside sealed programs a label occurs as a loan at most once; inside sealed programs it may occur any number of times, since the hole of a returned borrow sits in the fill of every borrowed argument (Figure 11).
3. **Loans are acyclic.** The relation “ loan_ℓ occurs in the content of borrow_m ” is acyclic, so owners is well defined.
4. **Captures are values.** Closures and Π -closures capture no borrow and no $\perp$, and sealed programs are closed.
5. **Arguments are exclusive.** At every call point, each argument is loan-free or is $\text{borrow}_\ell u$ with u loan-free. This is `[Close]`’s precondition, and `[Access]` maintains it.
6. **Values have their types.** For each binding $x : A \mapsto v$: $v = \perp$ (moved out or ended), or $v = \text{loan}_\ell$ and the content of borrow_ℓ has type A , or v has type A (a neutral having the type recorded for it). Every σ occurring in Ω , in a stored type or in the goal has a type in Δ , and every top-level name has an entry in Σ .

Lemma 2 states that the machine preserves conditions 1 to 5.

A.6 Notes on the definition

The prose rules (RULES v1.9) leave the following points open, or state them in a way that admits more than one reading. Each note gives the reading taken above, which is the checker’s unless the note says otherwise. Notes 1–4 record readings under which earlier rule sets or checker versions are unsound; the rules adopt the readings of notes 1 and 2, 3 and 4, and the checker rejects all four.

1. **The class of a local function.** v1.5 recorded a function’s erasure class “at `[Def]`”, from its codomain evaluated at its generic call. A local `fix` is checked by `[Def]` each time it is formed, including at each instance of a statement that contains it, and its codomain may depend on captured values:

```

U(n : Nat) : Typeo := match n { Z => Prop | S _ => Prop }
V(n : Nat) : U(n) := match n { Z => T | S _ => T }
LieL(n : Nat) : Id Nat (let h = λ(x : &Nat) : U(n) => (*x := S Z; V(n)); let c = Z;
h(&c); c) (S Z) := refl
BoomL : Eq Nat Z (S Z) := LieL(Z)

```

At `LieL`’s generic call, h ’s codomain is $[U(\sigma)]$, which is not a sort, so $h(\&c)$ runs, c ends as $S\ Z$, and the statement is T . At $n = Z$, h is formed again with codomain $U(Z) = \text{Prop}$, a sort, so $h(\&c)$ is erased, c stays Z , and $\text{LieL}(Z) : \text{Eq Nat } Z\ (S\ Z)$. The v1.6 checker accepts `BoomL`. Reading the class from the codomain *term* $(U(n)$: not syntactically a sort, declared sort `Typeo`) runs $h(\&c)$ on both paths.

2. **A stuck block of a type-valued match.** A stuck block is a call whose function’s codomain is the match’s type, so the call rule erases a block whose type is a sort, while the direct path does not erase the same match:

```

LieB(n : Nat) : Id Nat (let c = Z; let T = match n { Z => (c := S Z; T) | S _ => (c :=
S Z; T) }); c) Z := refl
BoomB : Eq Nat (S Z) Z := LieB(Z)

```

At the generic call the block (of type `Prop`) is erased, c stays Z , and the statement is T ; at $n = Z$ the match runs, c becomes $S\ Z$, and $\text{LieB}(Z) : \text{Eq Nat } (S\ Z)\ Z$. The v1.6 checker accepts `BoomB`. v1.7 first erased a block when its match would be erased; but the match’s type, inferred from the arms, is computed, and with arms $f(\&c)$ where f returns data and has codomain $V(Z)$ (which computes to T) the block was erased at the

generic call while the match ran at the instance (lean-checker P2, **BoomG**). Hence D40: a block is erased only when each arm is.

3. **Positivity.** v1.6 put no condition on field types; [Ind] requires first-order data (inductive types, **Unit**, $\times$). The v1.6 checker accepted `inductive Bad := Mk(f : $\Pi(x : \text{Bad}). \text{Empty}$) (with Empty := E(e : Empty)), whose eliminator into Eq Nat Z (S Z) is structural), then L(b : Bad) : Empty := match b { Mk(f) => f(b) }, the proof K(b : Bad) : Eq Nat Z (S Z) := absurd(L(b)), and K(Mk($\lambda(x : \text{Bad}) : \text{Empty} => L(x)$)) : Eq Nat Z (S Z), a closed proof never run.`
4. **Declared, not computed.** A `let` or sequence is a declared proof when its tail is, a match when the arm it takes is, and a variable when its binding is flagged; a type-valued tail does not make its context erased, so `let T = (c := S Z; F(&c)); ...` with `F` returning types keeps the write to `c`. Reading a variable as a proof because its *value* is $\star$ is unstable (lean-checker P3):

```
LieH(g :  $\Pi(y : \text{Nat}). V(Z)$ ) : Id Nat (let c = Z; let h = g(0); (c := S Z; h); c) (S
Z) := refl
BoomH : Eq Nat Z (S Z) := LieH(( $\lambda(y : \text{Nat}) : V(Z) => \text{refl}$ ))
```

`g`'s codomain `V(Z)` does not have declared sort **Prop**, so `g(0)` is data. At the generic call `h` holds a sealed program and the write is kept, so the statement is τ ; at the instance `g(0)` returns $\star$, a value-based reading erases `(c := S Z; h)`, and `LieH(...)` : `Eq Nat Z (S Z)`. Reading clause 4 by the type the typing judgement computes (here τ) is stable on its own, but mixing it with the syntactic class of calls is **BoomG** (note 2); so every clause is declared.

5. **Persistent generalisation.** A generalised sealed program is recorded in ρ , globally, so [Seal-stuck] replaces every later derivation of it, and the state is re-normalised whenever ρ grows. Matching up to $\equiv$ uses conversion, so a re-derivation that differs only in bound names is recognised. The generalised σ has the type of the matched place. The v1.6 checker discarded a record made on a private copy and rewound its fresh-name counter with the copy, so a later split reissues the same σ for a different sealed program (breaker-fresh-v16 X3, a closed proof of `Eq Nat 1 0`).
6. **Confinement.** A place *outlives* an erased term when its root is a position of the environment the term starts from; the exception for erased calls covers the borrows and moves that evaluate the call's arguments, and nothing else. The checker does not yet check confinement (in progress).
7. **Proofs are not run.** A proof call's arguments are evaluated and its [Call-type] and [Rec] checked, on a private copy; its body is never run ([T-Call-proof]), and the machine skips proofs altogether ([Erase-proof]). By P2 running and skipping agree.
8. **The order of [Access].** Loans are ended from the root of the place outward, then left to right inside its content. RULES fixes no order; by Theorem 8 (1) the resolution does not depend on it.
9. **A match on a constructor checks only the arm taken**, in typing as in the machine ([T-Match]).
10. **[Close]'s row is chosen by the declared codomain** (**Unit**, $\&\mathbf{T}$, anything else), as v1.7 and Lemma 7 (4) say, and so does the checker. A codomain that computes to **Unit** without being syntactically **Unit** gets the data row, and there is no η for **Unit**, so a statement such as `Id Unit (let c = *x; G(&c, Z)) ()`, with `G`'s codomain `UU(n)` and `UU(Z) = Unit`, needs induction on $*x$ rather than `refl` (the checker's `RowI`, `RowIInd`).
11. **A discarded value** (`t; u`) holding a live loan is an error, like a dying owned binding.
12. **The head guard covers neutral heads.** The v1.6 checker closed off a neutral-headed call even as the head call of a sealed program; for a codomain $\&\mathbf{T}$ with two or more borrow arguments this re-closed forever (breaker-fresh-v16 X5, a stack overflow).
13. **Stuck-block moves.** A whole variable is moved into a block if some arm, when checked, leaves it $\perp$, or if it is a borrow variable read or assigned as a whole (the checker's C19). Captures are ordered by the position of their root, then by length, and their arguments are evaluated in that order (breaker-fresh B10).
14. **Footprint.** A place rooted at a borrow variable contributes its owners even when it is only read, and a variable holding $\perp$ contributes nothing. Owners are positions, so a temporary can be an owner, when a type is formed while arguments are in flight.

15. **J**, **A**, **a**, **b** and **P** are type positions and **h** is a proof, so none of them runs in the machine; **J**'s value is **t**'s. The motive may have any sort (the checker's C12). **J** is a declared proof when its motive is syntactically a function into **Prop**, or when **t** is a declared proof.
16. **Universes**. $A \times B$ lives in $\text{Type}_0 \sqcup s_A \sqcup s_B$, $\&A$ and inductive types in $\text{Type}_\circ$, and Π is impredicative in **Prop** (the checker's C15). [T-And] requires both conjuncts to be propositions.
17. **Recursion** requires the declared decreasing parameter to have an inductive type, or to be a borrow of one; strict subterms range over all fields.
18. **Conversion** is the least relation of Section A.3 (RULES P1). Π -types are compared under binders by their captured values and code, not by evaluating them at generic arguments. There is no η rule, and no η for **Unit**.
19. **An unannotated non-tail match** needs its arms' types, each computed in its own refined state, to be convertible; an annotated one checks each arm against the annotation refined for that arm.
20. **Syntax**. **Nat** is presented as a declared type with field **1**; the checker builds it in, with `match p { Z => t | S y => u }`. The checker also has pair sub-places (**p.1**, **p.2** of a pair) and conveniences outside the core (`cong`, `trans`, `symm`, the ascription $(t : A)$, λ and $\rightarrow$).
21. **Known limitation: captured proofs and neutrals have no type**. `typeof` (Section A.1.4, item 9) has no case for $\star$ or a sealed program, and neither has the checker's `valType`. So a λ or Π whose body reads a captured proof, or captured neutral data, in a typed position cannot be checked: `let n = *x after AddM(&*x, 1)`, followed by `$\lambda(y : \text{Nat}) : \text{Nat} \Rightarrow n$` , is rejected. The fix is to record the types of captured values when a closure or Π -closure is formed. For the same reason captured values are never flagged as proofs.